

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

**DESARROLLO DE UN GUANTE INTELIGENTE
BASADO EN ARDUINO PARA LA
TRADUCCIÓN DE LENGUAJE DE SIGNOS**

AUTOR: DAVID PERIÑÁN CORBACHO

Puerto Real, septiembre de 2026

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

**DESARROLLO DE UN GUANTE INTELIGENTE
BASADO EN ARDUINO PARA LA
TRADUCCIÓN DE LENGUAJE DE SIGNOS**

TUTORAS: ALMUDENA MÁRQUEZ LOZANO Y MIRIAM
MENGÍBAR RODRÍGUEZ
AUTOR: DAVID PERIÑÁN CORBACHO

Puerto Real, septiembre de 2026

Declaración personal de autoría

David Perinán Corbacho con DNI 49563780E, estudiante del Grado en Ingeniería Informática en la Escuela Superior de Ingeniería de la Universidad de Cádiz, como autor de este documento académico titulado Guante inteligente traductor y presentado como Trabajo Final de Grado

DECLARO QUE

Es un trabajo original, que no copio ni utilizo parte de obra alguna sin mencionar de forma clara y precisa su origen tanto en el cuerpo del texto como en su bibliografía y que no empleo datos de terceros sin la debida autorización, de acuerdo con la legislación vigente. Asimismo, declaro que soy plenamente consciente de que no respetar esta obligación podrá implicar la aplicación de sanciones académicas, sin perjuicio de otras actuaciones que pudieran iniciarse. En Puerto Real, a septiembre de 2026

Fdo: David Perinán Corbacho

Agradecimientos

Quiero expresar mi agradecimiento a todas las personas que han contribuido, de una forma u otra, al desarrollo de este proyecto. En primer lugar, a mis padres, por el apoyo que me han brindado durante todo el proyecto. A mi padre, por enseñarme a diseñar y montar una placa PCB, así como las técnicas de soldadura necesarias para el desarrollo del hardware; y a mi madre, por su ayuda en la integración de los distintos componentes en el guante y por su colaboración en la búsqueda de recursos para aprender sobre la lengua de signos española.

Asimismo, quiero agradecer a mis tutoras su orientación, dedicación y paciencia a lo largo de todo el desarrollo del trabajo, así como la metodología y los consejos que me permitieron afrontar cada etapa del proyecto con una planificación adecuada. También agradezco a mis amigos su apoyo durante este proceso, tanto por ayudarme a mantener el equilibrio entre el trabajo y el tiempo de descanso como por las ideas y sugerencias que aportaron, especialmente en el ámbito del aprendizaje automático. A todos ellos, gracias por su confianza, su tiempo y su contribución para hacer posible este proyecto.

Resumen

El presente Trabajo de Fin de Grado aborda el desarrollo de un guante inteligente para la traducción de la lengua de signos española a texto, con el objetivo de facilitar la comunicación entre personas signantes y no signantes. La motivación surge de la necesidad de superar las barreras comunicativas a las que se enfrentan colectivos como personas con discapacidad auditiva, trastorno del espectro autista o parálisis cerebral, donde la comunicación oral puede verse limitada.

El sistema se compone de un guante sensorizado que integra cinco sensores de flexión para medir la curvatura de los dedos y una unidad de medida inercial para capturar la orientación y el movimiento de la mano. Todos los componentes electrónicos se centralizan en una placa Arduino personalizada, permitiendo un dispositivo compacto y ergonómico.

Para el reconocimiento de gestos se ha generado un conjunto de datos propio de 5860 muestras distribuidas en 31 clases debido a la inexistencia de bases de datos públicas adaptadas a la lengua de signos española. Se ha realizado un estudio comparativo entre seis algoritmos de aprendizaje automático, seleccionando *Random Forest* como modelo definitivo al alcanzar una precisión del 99,74 % y ofrecer el mejor equilibrio entre capacidad de clasificación y eficiencia computacional para el despliegue en tiempo real.

La aplicación de escritorio desarrollada en Python facilita dos modos de funcionamiento principales: la adquisición de datos para la creación de conjuntos de entrenamiento y la traducción de gestos en tiempo real. La comunicación con el guante puede realizarse mediante conexión USB o WiFi, proporcionando flexibilidad al usuario.

Los resultados obtenidos demuestran la viabilidad del enfoque propuesto, validando el sistema mediante pruebas unitarias, de integración y de sistema que verifican el cumplimiento de los requisitos funcionales y no funcionales establecidos. Este prototipo constituye una base sólida para futuras ampliaciones, como la incorporación de salida de voz, el despliegue autónomo en el microcontrolador o el desarrollo de una aplicación móvil que aumente su portabilidad y utilidad en entornos cotidianos.

Índice general

Índice de figuras	XI
-------------------	----

Índice de tablas	XIII
------------------	------

I Prolegómeno	1
----------------------	----------

1. Introducción	3
------------------------	----------

1.1. Motivación	3
---------------------------	---

1.2. Alcance y objetivos	4
------------------------------------	---

1.3. Glosario de términos	4
-------------------------------------	---

1.4. Organización del documento	5
---	---

2. Antecedentes	9
------------------------	----------

2.1. Introducción	9
-----------------------------	---

2.2. Estado de la técnica	10
-------------------------------------	----

2.2.1. Análisis de proyectos existentes	10
---	----

2.2.2. Hardware	21
---------------------------	----

2.2.3. Aprendizaje automático	22
---	----

2.2.4. Conjuntos de datos	23
-------------------------------------	----

3. Plan de gestión de proyecto	25
---------------------------------------	-----------

3.1. Metodología de desarrollo	25
--	----

3.2. Tecnologías	25
----------------------------	----

3.3. Planificación del proyecto	26
---	----

3.4. Organización	27
-----------------------------	----

3.5. Costes	28
-----------------------	----

3.5.1. Coste material	28
---------------------------------	----

3.5.2. Coste personal	28
---------------------------------	----

3.6. Riesgos	29
------------------------	----

3.7. Gestión de la configuración	30
II Desarrollo	31
4. Análisis del sistema	33
4.1. Requisitos del sistema	33
4.1.1. Requisitos funcionales	33
4.1.2. Requisitos no funcionales	33
4.1.3. Requisitos de información	35
4.2. Casos de uso	36
4.2.1. Modelos de casos de uso	36
4.2.2. Modelo conceptual	39
4.2.3. Modelo de interfaz de usuario	39
5. Diseño del sistema	43
5.1. Arquitectura del sistema	43
5.1.1. Diseño de alto nivel	43
5.1.2. Diseño detallado	45
5.2. Parametrización del software base	48
5.3. Diseño físico de datos	48
6. Montaje del guante	51
6.1. Descripción general	51
6.2. Componentes utilizados	51
6.3. Conexiones eléctricas y esquema del circuito	52
6.3.1. Diseño de la PCB	52
6.4. Fabricación y montaje del sistema	53
6.4.1. Fabricación de la PCB	53
6.4.2. Integración de los componentes en el guante	54
7. Aprendizaje de gestos	57
7.1. Objetivo de la comparación	57
7.2. Metodología de evaluación	57

7.2.1. Algoritmos de clasificación	58
7.2.2. Conjunto de datos	59
7.2.3. Optimización de hiperparámetros	60
7.2.4. Validación cruzada	60
7.2.5. Métricas de evaluación	61
7.3. Resultados obtenidos	62
8. Aplicación de escritorio	69
8.1. Descripción general	69
8.2. Arquitectura de la aplicación	69
8.3. Diseño de la interfaz de usuario	70
9. Pruebas del sistema	75
9.1. Estrategia	75
9.2. Pruebas unitarias	75
9.3. Pruebas de integración	76
9.4. Pruebas de sistema	79
9.4.1. Pruebas funcionales	79
9.4.2. Pruebas no funcionales	81
III Epílogo	85
10. Conclusiones	87
10.1. Objetivos alcanzados	87
10.2. Lecciones aprendidas	88
10.3. Trabajo futuro	89
Bibliografía	91
IV Anexos	95
A. Manual del desarrollador	97
A.1. Introducción	97

A.2. Preparación del entorno de trabajo	97
A.3. Consideraciones generales sobre el desarrollo	98
A.4. Instrucciones para construcción	99
B. Manual de usuario	101
B.1. Introducción	101
B.2. Instalación	101
B.3. Uso del sistema	101

Índice de figuras

2.1. Guante desarrollado [4].	12
2.2. Guante desarrollado [5].	13
2.3. Guante inteligente [6].	13
2.4. Guante con sensores triboeléctricos [7].	15
2.5. Integración en la realidad virtual.	15
2.6. GuanTELECO.	16
2.7. <i>Sign Glove</i>	17
2.8. Guante basado en KNN.	18
2.9. Guante de Nitusharaff.	19
2.10. Guante SignSpeak.	20
2.11. Guante de <i>heyitmeyo</i>	21
3.1. Ejemplo de circuito en Tinkercad.	26
3.2. Diagrama de Gantt.	27
4.1. Diagrama de casos de usos.	36
4.2. Diagrama UML.	39
4.3. Diagrama de navegación.	40
5.1. Diagrama de contexto.	44
5.2. Diagrama de contenedores.	44
5.3. Diagrama de componentes.	45
6.1. Esquema eléctrico del sistema desarrollado en KiCad.	52
6.2. Diseño de PCB desarrollado en KiCad.	53
6.3. Prototipo del guante.	54
7.1. <i>F1-score</i> de cada algoritmo.	64
7.2. Tiempo medio de entrenamiento de cada algoritmo.	65
7.3. Tiempo medio de predicción de cada algoritmo.	66

8.1. Pantalla del menú de lectura.	71
8.2. Pantalla de recolección de datos.	72
8.3. Pantalla del menú de traducción.	72
8.4. Pantalla de traducción en tiempo real.	73
8.5. Pantalla de error.	73
9.1. Resultado de ejecutar Pytest.	76

Índice de tablas

2.1. Comparativa de hardware de los guantes inteligentes.	22
2.2. Comparativa de modelos y lenguas de signos.	23
3.1. Tiempo dedicado a cada tarea para la realización de este trabajo. . . .	29
7.1. Algoritmos evaluados durante la comparación de modelos.	59
7.2. Ejemplo de tres instancias correspondientes a la letra A del conjunto de datos.	60
7.3. Accuracy y F1-score obtenidos para los algoritmos evaluados.	62
7.4. Tiempos de entrenamiento y predicción de los algoritmos evaluados. . .	63
A.1. Principales módulos de la aplicación.	98

Parte I

Prolegómeno

1. Introducción

1.1. Motivación

La comunicación es un proceso fundamental en la interacción humana que permite el intercambio de información entre individuos mediante distintos sistemas de representación. Este sistema se compone de varios elementos: emisor, receptor, mensaje, canal, contexto y código. Este proceso puede ser verbal o no verbal. En determinados casos, la comunicación verbal puede verse limitada o no ser posible, como ocurre en personas con discapacidad auditiva, dificultades del habla o trastornos del neurodesarrollo. En estos casos, la comunicación no verbal adquiere una especial relevancia.

Dentro de la comunicación no verbal, la lengua de signos constituye un sistema completo de comunicación que permite la expresión mediante signos manuales y expresiones corporales. Sin embargo, su uso está limitado a una parte reducida de la población, lo que puede generar barreras comunicativas entre personas signantes y no signantes. Según los datos de la Organización Mundial de la Salud, en febrero de 2025 se estimaron alrededor de 1.500 millones de personas con algún grado de pérdida auditiva, cifra que podría alcanzar los 2.500 millones para el año 2050. De este total, aproximadamente 430 millones requieren rehabilitación y cerca de 70 millones presentan sordera profunda. Esta situación implica importantes barreras de comunicación, especialmente debido al desconocimiento generalizado de la lengua de signos por parte de la población oyente.

Aunque la lengua de signos se asocia principalmente a personas con discapacidad auditiva, su aplicación puede extenderse a otros colectivos con dificultades en la comunicación oral. En el caso de las personas con Trastorno del Espectro Autista (TEA), la OMS estima que aproximadamente 1 de cada 127 personas presenta esta condición. Además, estudios recientes indican que alrededor del 40 % de las personas con TEA presentan autismo no verbal. De forma similar, algunas personas con parálisis cerebral pueden experimentar dificultades en la comunicación debido a alteraciones motoras, cognitivas o del habla. En España, se estima que hay una población de aproximadamente 120.000 personas con esta condición, mientras que en otros países, como México, se han registrado cifras superiores a 17 millones de personas.

Gracias al rápido avance de la tecnología, los dispositivos que anteriormente requerían un alto costo computacional son actualmente accesibles para un gran número de usuarios. Esto ha permitido la creación de microcontroladores compactos y de bajo consumo, con capacidad para implementar sistemas básicos de reconocimiento de patrones en tiempo real.

En este contexto, el desarrollo de un guante inteligente para la traducción de la lengua de signos surge de la necesidad de facilitar la comunicación entre las personas usuarias de este sistema y las personas no familiarizadas con él, especialmente en entornos donde no se dispone de intérpretes. El objetivo es mejorar la accesibilidad

y la comunicación de distintos colectivos, no únicamente de personas con discapacidad auditiva.

1.2. Alcance y objetivos

El alcance de este proyecto consiste en el desarrollo de un guante inteligente basado en una placa ESP32, sensores de flexión y un sensor inercial (IMU), capaz de capturar movimientos de la mano para la identificación de signos de la lengua de signos.

El sistema se centrará en el reconocimiento de signos realizados con una única mano, por lo que el vocabulario disponible estará limitado a este tipo de configuraciones.

El guante incorporará un modelo de *machine learning* encargado de clasificar los datos procedentes de los sensores y asociarlos a letras o signos previamente definidos. Este modelo se entrenará a partir de un conjunto de datos propio generado durante el desarrollo del proyecto.

Quedan fuera del alcance del proyecto aspectos como la traducción completa de frases en lengua de signos, la comunicación bidireccional en tiempo real o la adaptación a múltiples usuarios sin recalibración previa. No obstante, existen sistemas que han abordado la traducción de frases completas de lengua de signos [1]

A partir de este objetivo general, se establecen diversos objetivos específicos orientados al desarrollo completo del sistema. En primer lugar, se pretende capturar los datos de movimiento y flexión de la mano mediante los sensores integrados en el guante. Posteriormente, se diseña y genera un conjunto de datos propio a partir de la adquisición de signos, que servirá como base para el entrenamiento de un modelo de *machine learning* capaz de clasificar los distintos signos de la lengua de signos. Finalmente, el sistema deberá ser capaz de generar una salida en forma de texto a partir de la clasificación de los signos realizados.

1.3. Glosario de términos

- ASPACE: Asociación de Parálisis Cerebral.
- ASL (*American Sign Language*): Lengua de Signos Americana.
- CSV (*Comma-Separated Values*): Formato de archivo de texto utilizado para almacenar datos tabulares separados por comas.
- ESP32: Microcontrolador con conectividad WiFi y Bluetooth utilizado para la adquisición y transmisión de datos.
- INE: Instituto Nacional de Estadística.
- LSE: Lengua de Signos Española.

- *Machine learning*: Rama de la inteligencia artificial que permite a un sistema aprender patrones a partir de datos para realizar tareas específicas sin ser programado explícitamente para cada caso.
- MPU6050: Sensor inercial que integra un acelerómetro y un giroscopio de tres ejes.
- OMS: Organización Mundial de la Salud.
- PCB (*Printed Circuit Board*): Placa de circuito impreso utilizada para interconectar los componentes electrónicos del sistema.
- *Pitch*: Ángulo que representa la inclinación de la mano hacia delante o hacia atrás.
- *Roll*: Ángulo que representa la inclinación lateral de la mano.

1.4. Organización del documento

La documentación se reparte en 10 capítulos en total, distribuidos en tres partes: El prolegómeno, el desarrollo y el epílogo. A continuación, se describe brevemente el contenido de cada uno de ellos.

Prolegómeno

En el Capítulo 1 se introduce el contexto del trabajo, se expone la motivación y los objetivos del proyecto, se incorpora un glosario de términos y se presenta la organización de la memoria.

En el Capítulo 2 se presenta el estado del arte mediante el análisis de proyectos relacionados, las tecnologías utilizadas, los principales enfoques de aprendizaje automático y los conjuntos de datos disponibles para este tipo de sistemas.

En el Capítulo 3 se describe la metodología de desarrollo seguida, las tecnologías empleadas, la planificación y organización del proyecto, así como el análisis de costes, los riesgos identificados y la gestión de la configuración del sistema.

Desarrollo

En el Capítulo 4 se recogen la especificación de los requisitos funcionales y no funcionales del sistema, los requisitos de información, los casos de uso, el modelo conceptual y el diseño de la interfaz de usuario.

En el Capítulo 5 se presenta la arquitectura del sistema, tanto a nivel general como a nivel detallado, la parametrización del software base y el diseño físico de los datos empleados por la aplicación.

En el Capítulo 6 se describe el proceso de diseño y construcción del guante sensorizado, incluyendo los componentes utilizados, las conexiones eléctricas, el diseño de la PCB y la integración final de todos los elementos.

En el Capítulo 7 se presenta la comparación entre distintos algoritmos de aprendizaje automático, describiendo la metodología de evaluación, el conjunto de datos empleado, el proceso de optimización de hiperparámetros, la validación cruzada, las métricas utilizadas y los resultados obtenidos.

En el Capítulo 8 se describe el desarrollo de la aplicación utilizada para la adquisición de datos y la traducción de signos, detallando su arquitectura y el diseño de la interfaz de usuario.

En el Capítulo 9 se recoge la estrategia de pruebas seguida para validar el funcionamiento de la aplicación, incluyendo pruebas unitarias, de integración y de sistema, tanto funcionales como no funcionales.

Epílogo

En el Capítulo 10 se presentan las conclusiones del proyecto, valorando el grado de cumplimiento de los objetivos planteados y las principales lecciones aprendidas durante su desarrollo. Asimismo, se proponen posibles líneas de trabajo futuro orientadas a ampliar las funcionalidades del guante.

2. Antecedentes

En este capítulo se detalla la situación actual que origina el desarrollo o la mejora de un sistema informático. Asimismo, se describen las alternativas tecnológicas existentes.

2.1. Introducción

Para facilitar la comunicación de las personas con dificultades en el lenguaje oral, se han desarrollado los Sistemas Aumentativos y Alternativos de Comunicación (SAAC), que agrupan diferentes herramientas y estrategias destinadas a facilitar o sustituir la comunicación oral.

Su desarrollo comenzó a consolidarse durante el siglo XX, especialmente a partir de las investigaciones en educación especial y logopedia.

Los primeros SAAC se basaban en métodos no tecnológicos, como signos, tableros de comunicación, pictogramas y sistemas de símbolos visuales. Entre estos destacan los siguientes:

- Sistema simbólico gráfico-visual (BLISS): Desarrollado en 1949 por Charles K. Bliss. Emplea dibujos geométricos y elementos gráficos para la comunicación, junto con números, signos de puntuación, etc. Actualmente, dispone de más de 2000 símbolos.
- Programa de Comunicación Total – Habla Signada –: Desarrollado en 1980 por B.Schaeffer y colaboradores. Es el más utilizado recientemente. Se trata de un sistema bimodal que combina el lenguaje oral con signos manuales.
- Sistema Pictográfico de Comunicación (SPC): desarrollado en 1981 por Mayer y Johnson. Basado en pictogramas sencillos y de fácil interpretación, utilizados para facilitar la comunicación funcional.
- *Minspeak*: Desarrollado en 1982 por Bruce Baker para agilizar los procesos de comunicación, optimizando el tiempo necesario para emitir un mensaje mediante el uso de iconos (fijados entre el logopeda y el paciente).
- Sistema de Comunicación por Intercambio de Imágenes (PECS): Desarrollado en 1994 por Bondy y Frost con el fin de ayudar a niños con TEA en la adquisición de destrezas para la comunicación funcional. Se basa en el intercambio de imágenes entre el niño y las personas de su entorno.

A partir de estos sistemas iniciales, los SAAC han evolucionado hacia soluciones con ayuda tecnológica que incorporan dispositivos electrónicos para facilitar la comunicación. Este tipo de sistemas permite la generación de voz sintética, la organización dinámica del vocabulario y la creación de mensajes de forma más flexible [2].

Entre los sistemas de alta tecnología más representativos se encuentran los siguientes: En primer lugar, los comunicadores electrónicos con salida de voz, como los dispositivos dedicados de Tobii Dynavox. Se trata de SAAC que permiten al usuario comunicarse mediante la selección de símbolos, palabras o frases en una pantalla, que posteriormente son convertidas en voz sintética. Por otro lado, los sistemas basados en software de comunicación aumentativa, como Grid 3, que permiten la construcción de mensajes mediante símbolos, texto y síntesis de voz en dispositivos electrónicos.

La evolución de los SAAC y de las tecnologías ha permitido el desarrollo de sistemas de comunicación cada vez más avanzados e interactivos. Dentro de este contexto, surgen los guantes traductores de lengua de signos, dispositivos orientados a interpretar los movimientos de los dedos y de la mano, y transformarlos en texto o voz.

2.2. Estado de la técnica

Antes de desarrollar el sistema propuesto, se ha realizado un estudio de soluciones existentes relacionadas con la lengua de signos mediante guantes inteligentes. Este análisis permite identificar el estado actual de la tecnología y las principales aproximaciones utilizadas en este tipo de sistemas. La mayoría de las soluciones analizadas emplean sensores de flexión y sensores inerciales, ya que permiten capturar de forma adecuada los movimientos de los dedos y de la mano.

2.2.1. Análisis de proyectos existentes

Haris Bin Amir: Random Forest con fusión flex+IMU

El proyecto desarrollado por Haris Bin Amir [3] consiste en un guante inteligente para la traducción del lenguaje de signos, basado en sensores de flexión y un sensor inercial MPU6050. El sistema utiliza un microcontrolador tipo Arduino Nano como unidad principal de procesamiento, encargado de recoger y procesar las señales provenientes de los sensores.

A nivel de hardware, el sistema integra sensores de flexión en los dedos para medir el grado de curvatura, junto con el módulo MPU6050, que permite obtener información sobre la aceleración y la orientación de la mano. Esta combinación permite capturar tanto la posición de los dedos como el movimiento global de la mano, mejorando la discriminación entre signos similares.

En cuanto al modelo de *machine learning*, el sistema se basa en técnicas de aprendizaje supervisado para la clasificación de signos a partir de datos procedentes de los sensores. Para ello, se utilizaban conjuntos de datos etiquetados que representan diferentes signos de la lengua de signos.

El proceso de entrenamiento se realiza mediante una partición del conjunto de datos en un 80 % para entrenamiento y un 20 % para prueba, aplicando además ajuste de hiperparámetros mediante *GridSearchCV* con validación cruzada de tres particiones.

Este enfoque permite optimizar el rendimiento de los modelos, entre los que se incluyen *Random Forest*, *XGBoost* y redes neuronales implementadas con *TensorFlow* o *Keras*. El sistema evalúa el rendimiento mediante métricas de precisión, alcanzando valores cercanos al 97 % en el conjunto de prueba en los mejores casos.

Como limitación principal, el sistema está orientado al lenguaje de signos ASL y depende de un conjunto de datos específico generado para el entrenamiento, lo que reduce su generalización a otros lenguajes de signos o a usuarios sin recalibración previa.

Urav Dalal: BiLSTM con sensor flex+IMU

El trabajo presentado en el artículo [4] propone un sistema de guante inteligente para la traducción de lengua de signos, basado en el uso de un microcontrolador Arduino MEGA 2560. El sistema utiliza sensores de flexión y un sensor inercial MPU6050. La adquisición de datos se realiza mediante conexión I2C para el giroscopio y entradas analógicas para los sensores de flexión.

En cuanto al modelo de *machine learning*, el sistema emplea una red neuronal recurrente de tipo bidireccional LSTM (BiLSTM¹) para la clasificación de signos. El modelo utiliza una función de activación *Softmax*² para la clasificación multiclase y se entrena mediante la función de pérdida *Sparse Categorical Crossentropy*³. El entrenamiento se realiza durante 10 épocas con un tamaño de lote de 31, obteniéndose una precisión del 84.59 % en entrenamiento y del 82 % en datos no vistos, con un valor de ROC AUC⁴ de 0.90.

El conjunto de datos utilizado es propio y privado, basado en ASL, y compuesto por 34 clases que incluyen letras del alfabeto y palabras comunes. Cada muestra está formada por 11 características procedentes de los sensores. La captura de datos se realiza registrando cada signo durante tres segundos mediante un *script* de Python conectado al Arduino, almacenando las señales en archivos CSV. El dataset incluye tanto signos estáticos como dinámicos.

¹Modelo de red neuronal que procesa secuencias en ambos sentidos (hacia delante y hacia atrás), mejorando la captura de dependencias temporales.

²Función de activación que convierte salidas en probabilidades normalizadas para clasificación multiclase

³Función de pérdida utilizada en clasificación multiclase cuando las etiquetas son enteros.

⁴Métrica que mide la capacidad de un modelo para distinguir entre clases, donde 1 representa clasificación perfecta.

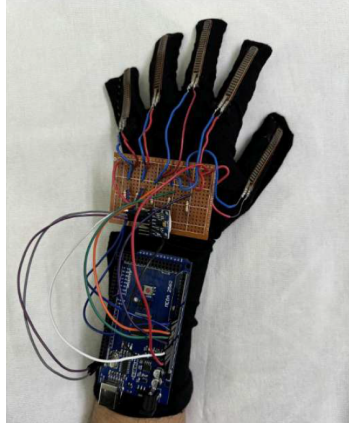


Figura 2.1: Guante desarrollado [4].

Ang Ji: Attention-BiLSTM con 16 sensores inerciales

El trabajo presentado en el artículo [5] propone un sistema de guante inteligente para el reconocimiento de lengua de signos basado en una red densa de sensores inerciales distribuidos a lo largo de la mano. El sistema se compone de un guante equipado con 16 unidades de adquisición de movimiento (MANs⁵), cada una integrada por un sensor MPU9250 que combina acelerómetro, giroscopio y magnetómetro. La distribución de sensores incluye tres unidades por dedo y una adicional en el dorso de la mano para capturar el movimiento articular. El sistema está controlado por un microcontrolador STM32F407VGT6⁶ y utiliza un módulo ESP8266⁷ para la transmisión inalámbrica de datos mediante WiFi, operando a una frecuencia de muestreo de 100 Hz.

En cuanto al modelo de *machine learning*, el estudio compara múltiples algoritmos de clasificación, incluyendo árboles de decisión, *Support Vector Machine* (SVM) con kernel RBF, *K-Nearest Neighbors* (KNN) y *Random Forest*. Además, se propone un modelo de *deep learning* basado en *Attention-BiLSTM*, compuesto por dos capas bidireccionales LSTM, capas de *dropout* para regularización, un mecanismo de atención y una capa completamente conectada para la salida. El modelo utiliza un *learning rate* de 0.001, un tamaño de lote de 256 y 100 épocas de entrenamiento. Los resultados muestran que el modelo de *deep learning* propuesto alcanza la mayor precisión con un 98.85 %, seguido de *Random Forest* con un 97.58 %.

El conjunto de datos utilizado es un dataset privado propio compuesto por 2219 muestras correspondientes a 20 signos dinámicos de la lengua de signos china. Cada muestra se registra durante 4 segundos e incluye información de aceleración, velocidad angular y orientación procedente de los sensores inerciales. El conjunto de datos fue recopilado por cinco participantes y posteriormente procesado hasta obtener representaciones de alta dimensionalidad. Además, el estudio valida el rendimiento del sistema comparándolo con múltiples conjuntos de datos públicos de reconocimiento de signos.

⁵Motion Acquisition Node: unidad de adquisición de movimiento basada en sensores inerciales

⁶Microcontrolador de 32 bits basado en arquitectura ARM Cortex-M4.

⁷Módulo de comunicación WiFi de bajo coste ampliamente utilizado en sistemas embebidos.

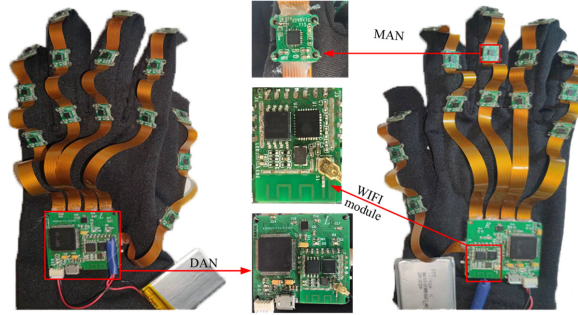


Figura 2.2: Guante desarrollado [5].

Swaroop Gudi: CNN desplegada en Android para lengua india (ISL)

El trabajo presentado [6] propone un sistema de guante inteligente para la detección de lengua de signos basado en sensores de flexión y comunicación inalámbrica. El sistema utiliza un microcontrolador Arduino Nano junto con sensores de flexión, uno por cada dedo, encargados de capturar la curvatura de los dedos durante la realización de los signos. La comunicación de los datos se realiza mediante un módulo *Bluetooth* HC-05, permitiendo su transmisión a una aplicación Android encargada de la interpretación y salida del sistema.

En cuanto al modelo de *machine learning*, el sistema emplea una red neuronal convolucional (CNN) para la clasificación de signos de lengua de signos. La arquitectura del modelo está compuesta por varias capas convolucionales encargadas de la extracción de características, seguidas de capas de *pooling* para la reducción de dimensionalidad y capas densas completamente conectadas con activación *ReLU*. La salida del modelo se realiza mediante una capa *Softmax* para la clasificación multiclase. Para su implementación en entornos móviles, se optimiza mediante *TensorFlow Lite*, reduciendo la latencia y el consumo computacional.

El sistema se entrena con un conjunto de datos propio basado en la lengua de signos india (ISL), centrado principalmente en el reconocimiento de signos del alfabeto. El modelo alcanza una precisión media superior al 90% y una latencia inferior a 200 milisegundos por predicción. Sin embargo, los autores señalan la restricción del conjunto de datos como una limitación, ya que no contempla suficiente variabilidad entre diferentes usuarios.

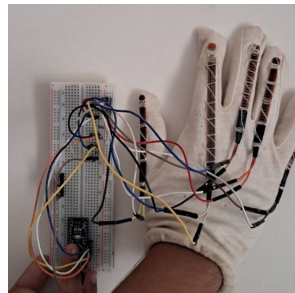


Figura 2.3: Guante inteligente [6].

Feng Wen: sensores triboeléctricos autoalimentados + RV

El trabajo presentado en *Nature Communications* [7] propone un sistema de guante inteligente basado en sensores triboelectrónicos (TENG) para el reconocimiento de la lengua de signos y la comunicación en entornos de realidad virtual. El sistema está compuesto por un guante con 15 sensores triboeléctricos distribuidos en los dedos, la palma y la muñeca, lo que permite capturar de forma detallada tanto los movimientos de flexión como la dinámica global de la mano. Estos sensores están fabricados con materiales como Ecoflex, nitrilo arrugado y textiles conductores, y presentan la particularidad de ser autoalimentados mediante el efecto triboeléctrico. El sistema utiliza un microcontrolador Arduino MEGA 2560 y se integra con un módulo de comunicación TCP/IP para su interacción con entornos de realidad virtual.

En cuanto al modelo de *machine learning*, el sistema emplea una red neuronal convolucional unidimensional (CNN 1D) para la clasificación de las señales procedentes de los sensores. El modelo está configurado con múltiples capas convolucionales (64 filtros y kernel de tamaño 5), entrenado durante 50 épocas. Además, el estudio propone dos estrategias de reconocimiento. La primera consiste en un enfoque sin segmentación donde se procesan señales completas de palabras o frases. La segunda emplea una segmentación mediante ventana deslizante, que divide las señales en fragmentos para su posterior clasificación jerárquica. Este enfoque permite reconocer tanto palabras individuales como frases completas, incluso en casos no vistos previamente, mediante la combinación de unidades conocidas.

El sistema se valida con un conjunto de datos propio basado en la lengua de signos americana (ASL), compuesto por 50 palabras y 20 frases, con 100 muestras por cada clase. El dataset incluye señales de 15 canales correspondientes a los sensores triboeléctricos y se encuentra disponible públicamente en Harvard Dataverse⁸. El modelo alcanza una precisión del 91.3 % en palabras y hasta un 95 % en frases, destacando además la capacidad del sistema para reconocer estructuras completas de lenguaje y no únicamente signos aislados.

Como característica destacable, el estudio introduce la integración con entornos de realidad virtual mediante comunicación bidireccional, lo que amplía significativamente las posibilidades de interacción humano-máquina en este tipo de sistemas (ver Figura 2.5).

⁸<https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/7KJWV3>



Figura 2.4: Guante con sensores triboeléctricos [7].

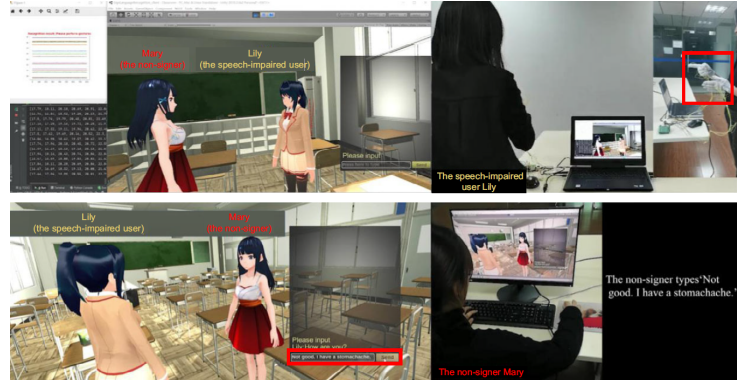


Figura 2.5: Integración en la realidad virtual.

GuanTELECO: sin ML, reglas por umbrales (LSE)

El proyecto GuanTELECO⁹ propone un guante traductor de lengua de signos de bajo coste basado en sensores inerciales y una arquitectura embebida. El sistema utiliza un microcontrolador ESP32 Heltec, que integra una pantalla OLED, junto con seis sensores MPU6050 distribuidos en la mano (ver Figura 2.6) para capturar la orientación y el movimiento de la mano. La salida del sistema se realiza mediante un módulo *DFPlayer Mini* conectado a un altavoz, permitiendo la reproducción de audio correspondiente a los signos destacados.

En cuanto al procesamiento de la información, el sistema no emplea técnicas de *machine learning* ni *deep learning*, sino que se basa en un conjunto de reglas determinísticas y umbrales fijos. Estos umbrales permiten clasificar el estado de cada dedo y la orientación global de la mano, estableciendo patrones predefinidos para la identificación de letras del alfabeto de la lengua de signos. Además, el sistema incorpora un mecanismo de validación temporal que requiere la estabilidad del signo durante aproximadamente un segundo para confirmar la detección, reduciendo los falsos positivos.

El sistema está orientado al reconocimiento del alfabeto de la lengua de signos española (LSE), funcionando como un prototipo inicial de bajo coste. No se utiliza ningún dataset de entrenamiento, ya que la clasificación se realiza mediante reglas fijas definidas manualmente.

Como principales características destacables, el sistema prioriza la reducción de costes y la autonomía, integrando una batería propia y un diseño compacto basado en PCB personalizada. Sin embargo, presenta limitaciones importantes en escalabilidad y flexibilidad.

⁹<https://blogs.upm.es/tfb/wp-content/uploads/sites/813/2021/05/Memoria-GuanTELECO.pdf>



Figura 2.6: GuanTELECO.

SignGlove: sin ML, umbrales + App Inventor (ASL)

El proyecto *Sign Glove*¹⁰ propone un sistema de guante inteligente para la traducción de lengua de signos, basado en sensores de flexión y una arquitectura de bajo costo. El sistema utiliza un microcontrolador Arduino Uno Rev3 encargado de leer las señales procedentes de cinco sensores de flexión, uno por cada dedo, que varían su resistencia en función del grado de curvatura de la mano. Estas señales se procesan y se envían a una aplicación móvil desarrollada con *MIT App Inventor 2*, la cual se encarga de mostrar y reproducir la salida correspondiente a los signos detectados.

En cuanto al procesamiento de la información, el sistema no utiliza técnicas de *machine learning* ni *deep learning*, sino que utiliza umbrales fijos aplicados a las lecturas analógicas de los sensores. Cada letra del alfabeto de la lengua de signos se define mediante un conjunto de condiciones *if-else* que comparan los valores de cada sensor con rangos predefinidos.

El sistema está orientado al reconocimiento del alfabeto completo de la lengua de signos americana (ASL). Como salida, la aplicación móvil muestra el texto correspondiente y proporciona síntesis de voz, facilitando la comunicación con personas no signantes.

Como principal limitación, el sistema presenta una baja capacidad de generalización, ya que depende estrictamente de umbrales predefinidos y no incorpora mecanismos de aprendizaje.

¹⁰<https://projecthub.arduino.cc/rushilsaraswat/sign-glove-780325>

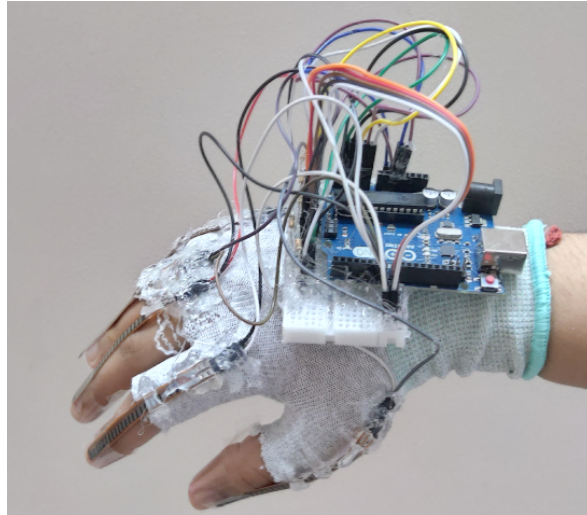


Figura 2.7: *Sign Glove*.

Gill et al.: KNN con visualización 3D en web

El proyecto *Smart Sign Language Translator Glove*¹¹ propone un sistema de guante inteligente para la traducción de lengua de signos basado en sensores de flexión y sensores inerciales, utilizando como unidad de procesamiento un microcontrolador ESP32 con conectividad WiFi integrada. El sistema incorpora cinco sensores flex, uno por cada dedo, junto con un sensor inercial MPU6050 encargado de capturar la orientación y el movimiento tridimensional de la mano. Las señales se adquieren mediante un divisor de tensión formado por resistencias de $1k\Omega$, permitiendo la lectura de los cambios de resistencia asociados a la flexión de los dedos.

En cuanto al modelo de *machine learning*, el sistema emplea un algoritmo de *K-Nearest Neighbor* (KNN) para la clasificación de signos. Los datos se recogen en ventanas temporales de aproximadamente tres segundos para capturar la dinámica completa del signo y, posteriormente, se normalizan para mejorar la robustez del modelo frente a variaciones entre usuarios. La clasificación se realiza en función de la distancia entre muestras en el espacio de características, asignando cada signo a la clase más cercana según el conjunto de entrenamiento.

El sistema utiliza un conjunto de datos propio generado durante la fase de adquisición (no se ha especificado ningún idioma), donde las señales de los sensores se almacenan y etiquetan manualmente en función del signo realizado. Este enfoque permite adaptar el modelo a signos personalizados sin necesidad de bases de datos públicas.

Como característica destacable, el sistema incluye una visualización en tiempo real interactiva mediante un modelo 3D de la mano en un navegador web, sincronizado con los datos del guante, además de salida textual y conversión a voz mediante la *Web Speech API*. Esto permite una interacción más intuitiva entre el usuario y el sistema.

¹¹<https://github.com/Gill003/Smart-Sign-Language-Translator-Glove>

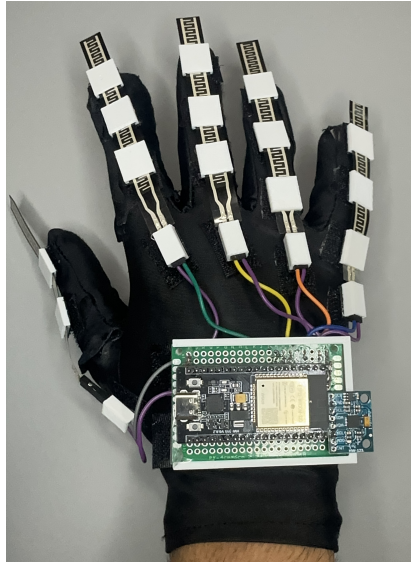


Figura 2.8: Guante basado en KNN.

Nitusharaff: comparativa de 5 clasificadores (MLP 98.7%)

El proyecto *Sign Language Glove*¹² propone un sistema de guante inteligente para la traducción de lengua de signos americana (ASL) con una arquitectura bidireccional: permite tanto la conversión de signos a voz/texto como la conversión de voz a vídeos de signos. El guante incorpora múltiples sensores de flexión para medir la curvatura de los dedos y la palma, junto con un acelerómetro para capturar el movimiento asociado a ciertos signos. Como unidad de procesamiento se utiliza un microcontrolador Arduino (modelo no especificado), y durante la fase de adquisición de datos se utiliza el software PLX-DAQ para volcar las lecturas de los sensores directamente a una hoja de cálculo.

En cuanto al software, el sistema se compone de dos partes diferenciadas. El código embebido en Arduino gestiona la lectura de los sensores, mientras que el backend se desarrolla en Python utilizando la biblioteca *Scikit-learn* para la implementación de los algoritmos de *machine learning*. Con el objetivo de evaluar el rendimiento de distintos clasificadores, el estudio compara los algoritmos *Support Vector Machine (SVM)*, árboles de decisión, *Random Forest*, *Naïve Bayes* y perceptrón multicapa, obteniendo precisiones del 96 %, 96 %, 83 %, 90 % y 98,7 %, respectivamente.

El sistema utiliza un conjunto de datos propio generado durante la fase de adquisición, que incluye todas las letras del alfabeto ASL y un conjunto de palabras de uso frecuente. El dataset consta de aproximadamente 13.000 muestras almacenadas y etiquetadas manualmente.

¹²<https://github.com/nitusharaff/Sign-Language-Glove>

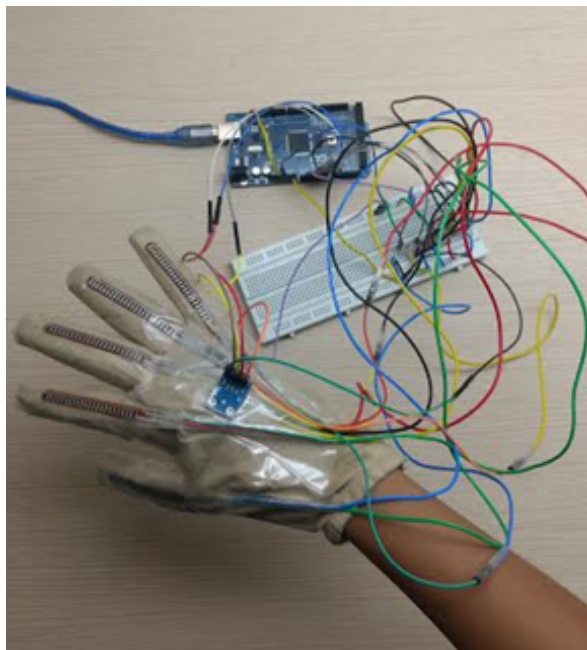


Figura 2.9: Guante de Nitusharaff.

Un-Mute: ESP32 con TensorFlow y Scikit-learn

El proyecto Un-Mute¹³ propone un sistema de guante inteligente para la traducción de lengua de signos a texto o voz en tiempo real. El sistema utiliza como unidad de procesamiento un microcontrolador ESP32, encargado de la adquisición y transmisión de datos. El guante integra cinco sensores de flexión, uno por cada dedo, junto con una unidad MPU6050, utilizada para capturar la orientación y el movimiento de la mano. Además, el sistema incorpora una aplicación externa para el procesamiento y visualización de los resultados obtenidos.

En cuanto al modelo de *machine learning*, el proyecto emplea técnicas de aprendizaje supervisado implementadas mediante librerías como *Scikit-learn*, *TensorFlow* y *Keras*. Las señales provenientes de los sensores se transforman en vectores de características utilizados para el entrenamiento de modelos de clasificación de signos, permitiendo reconocer diferentes signos realizados por el usuario.

El sistema utiliza un dataset propio, generado a partir de la captura directa de signos realizados con el guante. Las muestras son almacenadas y etiquetadas manualmente según el signo correspondiente, permitiendo entrenar los modelos de clasificación. El sistema está orientado a la traducción de la lengua de signos americana.

SignSpeak: LSTM, GRU y Transformer con dataset público

El proyecto *SignSpeak*¹⁴ propone un sistema de guante inteligente de bajo costo para la traducción de la lengua de signos americana (ASL) a voz en tiempo real. El

¹³https://github.com/MakersAsylumIndia/IS2025_Un-Mute

¹⁴<https://github.com/adityamakkar000/SignSpeak>

hardware del guante integra cinco sensores de flexión, uno por cada dedo, encargados de medir la curvatura de cada dedo. Como unidad de procesamiento se emplea un microcontrolador Arduino MEGA 2560, que adquiere las señales de los sensores a una frecuencia de 36 Hz y las transmite por puerto serie a una base de datos.

En cuanto al modelo de *machine learning*, el sistema aborda el problema como una clasificación de series temporales. Se implementan y comparan tres arquitecturas de redes neuronales: Redes LSTM (*Long Short-Term memory*) apiladas, redes GRU (*Gated Recurrent Unit*) apiladas y modelos basados en *Transformers*. Tras la evaluación de las tres arquitecturas, el mejor modelo alcanza una precisión del 92 % en la clasificación.

El sistema utiliza un conjunto de datos público, denominado *SignSpeak dataset*. El dataset consta de 7.200 muestras distribuidas en 36 clases (letras de la A a la Z y números del 1 al 10). Todas las muestras han sido etiquetadas manualmente.

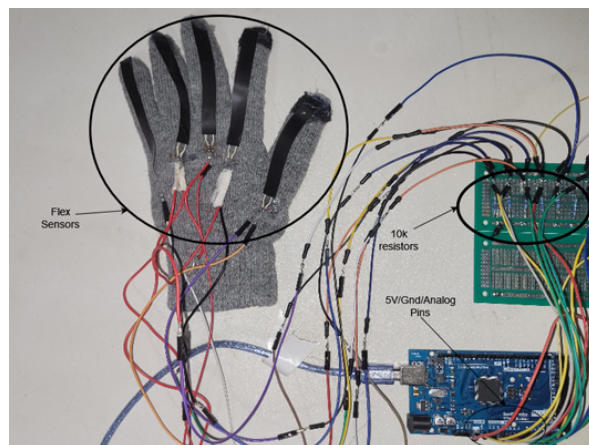


Figura 2.10: Guante SignSpeak.

Smart-glove: Random Forest desplegado en ESP32 y RPi

El proyecto *Smart-glove*¹⁵ propone un sistema de guante inteligente para la traducción de la lengua de signos americana (ASL) a texto y voz. El hardware del guante integra cinco sensores de flexión, uno por cada dedo, junto con un módulo MPU6050. Como unidad de adquisición se emplea un microcontrolador ESP32, y el sistema permite dos arquitecturas de despliegue. En la primera, el modelo de *machine learning* se ejecuta en una Raspberry Pi 4 Model B para un procesamiento más potente. En la segunda, el modelo se despliega directamente en el ESP32 mediante la librería *Micromlegen*.

En cuanto al modelo de *machine learning*, el sistema emplea un algoritmo de *Random Forest* para la clasificación de signos. El modelo se entrena con un conjunto de datos público disponible en Figshare¹⁶, que contiene las señales de los sensores correspondientes a los signos de ASL.

¹⁵<https://github.com/heyitsmeyo/Smart-glove>

¹⁶<https://figshare.com/articles/dataset/ASL-Sensor-Dataglove-Dataset.zip/20031017?file=35776586>



Figura 2.11: Guante de *heyitmeyo*.

WaniaKhance: comparativa de 6 clasificadores (ASL)

El proyecto *Smart Glove - Sign Language Translator*¹⁷ propone un sistema de guante inteligente para la traducción de lengua de signos en tiempo real, galardonado con el tercer premio al mejor proyecto final de carrera en la Universidad de Bahria. El hardware del guante integra sensores de flexión para determinar la curvatura de cada dedo, junto con un giroscopio. Como unidad de procesamiento, se emplea un microcontrolador Arduino que recibe los datos analógicos de los sensores, y un módulo WiFi permite la comunicación inalámbrica entre el guante y el ordenador.

En cuanto al modelo de *machine learning*, el sistema implementa y compara seis algoritmos de clasificación: KNN, SVM, árboles de decisión, *Naïve Bayes*, *Random Forest* y regresión logística. Todo el entrenamiento se realiza con la librería *Scikit-learn* en Python. Según el repositorio, se alcanza una precisión que supera el 90 % en la predicción de signos. El código se organiza en tres módulos principales: `arduino.ino` para la lectura en tiempo real, `data prediction using ML.py` para el entrenamiento de los modelos, y `loading models.py` para la aplicación de los modelos guardados sobre datos en tiempo real.

El sistema utiliza un conjunto de datos propio recolectado manualmente en la universidad, disponible en el repositorio. El sistema está orientado a la lengua de signos americana (ASL).

2.2.2. Hardware

Los proyectos revisados emplean una variedad de microcontroladores, sensores y protocolos de comunicación.

En cuanto a las unidades de procesamiento, se utilizan desde placas de bajo costo como Arduino Nano y Arduino Uno, hasta plataformas como Arduino MEGA 2560, ESP32 (con conectividad WiFi integrada) y, en el extremo superior, microcontroladores

¹⁷<https://github.com/WaniaKhance/Smart-Glove-Sign-Language-Translator>

de 31 bits como el STM32. Algunos sistemas optan por arquitecturas híbridas que combinan un ESP32 para adquisición con una Raspberry Pi para el procesamiento del modelo.

Sobre los sensores, la configuración estándar consiste en cinco sensores de flexión, uno para cada dedo, que miden el grado de curvatura [8, 9]. Los sensores inerciales también han sido empleados en guantes instrumentados para capturar el movimiento y la orientación de la mano durante la realización de signos [10]. Como alternativas más avanzadas, existen sistemas con sensores triboeléctricos autoalimentados o con múltiples unidades inerciales distribuidas por toda la mano.

Respecto a la comunicación, predominan tres enfoques: la transmisión por puerto serie a un ordenador, el Bluetooth hacia aplicaciones móviles y el WiFi para enviar los datos a servidores locales o interfaces web.

Una minoría de los sistemas integra la salida directamente en el guante mediante pantallas OLED o altavoces.

Proyecto	Placa	Sensores	Comunicación
Haris Bin Amir [3]	Arduino Nano	Flex + MPU6050	N/D
Urav Dalal [4]	Arduino Mega	Flex + MPU6050	Puerto serie
Ang Ji [5]	STM32	16x MPU9250	WiFi (ESP8266)
Swaroop Gudi [6]	Arduino Nano	Flex	Bluetooth (HC-05)
Feng Wen [7]	Arduino Mega	15 TENG	TCP/IP
GuanTELECO ¹⁸	ESP32 Heltec	6x MPU6050	Salida directa
SignGlove ¹⁹	Arduino Uno	Flex	Bluetooth
Gill ²⁰	ESP32	Flex + MPU6050	WiFi
Nitusharaff ²¹	Arduino Mega	Flex + acelerómetro	Puerto serie
Un-Mute ²²	ESP32	Flex + MPU6050	N/D
SignSpeak ²³	Arduino Mega	Flex	Puerto serie
Smart-glove ²⁴	ESP32 / RPi 4	Flex + MPU6050	WiFi
WaniaKhance ²⁵	Arduino	Flex + giroscopio	WiFi

Tabla 2.1

Comparativa de hardware de los guantes inteligentes.

2.2.3. Aprendizaje automático

Los sistemas revisados emplean principalmente dos grandes familias de técnicas: métodos clásicos de aprendizaje supervisado y redes neuronales profundas.

Entre los métodos clásicos, los más utilizados son *Random Forest*, KNN, SVM y los árboles de decisión. Estas técnicas destacan por su bajo coste computacional, lo que permite su despliegue en microcontroladores de gama baja como Arduino Nano o ESP32 mediante librerías de inferencia ligera. *Random Forest* es el más recurrente, ofreciendo un buen equilibrio entre precisión y eficiencia [11].

Entre las redes neuronales, se emplean principalmente redes convolucionales (CNN) para la extracción de características espaciales y redes recurrentes como LSTM o GRU para el procesamiento de series temporales. Algunos sistemas avanzados incorporan arquitecturas bidireccionales (BiLSTM) o mecanismos de atención (*Attention-BiLSTM*)

que mejoran la captura de dependencias temporales. Estos modelos alcanzan mayores precisiones, pero requieren una mayor capacidad de procesamiento y volúmenes de datos más extensos, por lo que suelen ejecutarse en ordenadores externos o en plataformas como Raspberry Pi.

En el extremo opuesto, algunos sistemas prescinden completamente de técnicas de aprendizaje automático, utilizando umbrales fijos y reglas determinísticas. Este enfoque es el más sencillo y de menor coste computacional, pero presenta una capacidad de generalización muy limitada y es sensible a las variaciones entre usuarios.

Por último, en cuanto al rendimiento, la precisión es la medida más reportada. Los modelos basados en *deep learning* alcanzan valores entre el 82 % y el 99 %, mientras que los métodos clásicos se sitúan entre el 90 % y el 98 %.

Proyecto	Modelo	Precisión	Lengua
Haris Bin Amir [3]	Random Forest	97 %	ASL
Urav Dalal [4]	BiLSTM	82 %	ASL
Ang Ji [5]	Attention-BiLSTM	98,85 %	China
Swaroop Gudi [6]	CNN	>90 %	ISL
Feng Wen [7]	CNN 1D	91.3 %	ASL
GuanTELECO ²⁶	Umbrales	No especificada	LSE
SignGlove ²⁷	Umbrales	No especificada	ASL
Gill ²⁸	KNN	No especificada	No especificada
Nitusharaff ²⁹	Perceptrón multicapa	98.7 %	ASL
Un-Mute ³⁰	Scikit-learn / TF	No especificada	ASL
SignSpeak ³¹	LSTM/GRU/Transformer	92 %	ASL
Smart-glove ³²	Random Forest	No especificada	ASL
WaniaKhance ³³	RF/SVM/KNN	>90 %	ASL

Tabla 2.2

Comparativa de modelos y lenguas de signos.

2.2.4. Conjuntos de datos

La mayoría de los proyectos emplea conjuntos de datos propios, generados y etiquetados manualmente por los autores, lo que limita la reproducibilidad y comparación entre sistemas. Solo unos pocos utilizan conjuntos de datos públicos, como el guante triboeléctrico (*Harvard Dataverse*), SignSpeak (*Harvard Dataverse*) y Smart-Glove (*Figshare*).

En cuanto a los idiomas de los signos, el inglés (ASL) es mayoritario. Aunque hay excepciones, como la publicada por Swaroop Gudi, que utiliza ISL; el proyecto de Ang Ji utiliza la lengua china y GuanTELECO el español (LSE).

Aunque existen conjuntos públicos de datos sobre lengua de signos, en este proyecto se creará un conjunto propio centrado en LSE, utilizando como referencia los ya existentes [3].

3. Plan de gestión de proyecto

En esta sección se describen todos los aspectos relativos a la gestión del proyecto: metodología, organización, costos, planificación, riesgos y aseguramiento de la calidad.

3.1. Metodología de desarrollo

El presente proyecto engloba tanto el desarrollo de hardware (diseño y montaje del guante sensorizado) como el desarrollo de software (adquisición de datos, creación del conjunto de datos, entrenamiento de modelos de *machine learning* y traducción de signos a texto).

Debido a la naturaleza del proyecto, se ha optado por una metodología de desarrollo incremental, dividiendo el proceso en varias fases.

En primer lugar, se llevó a cabo una investigación centrada en el análisis del estado del arte, el estudio de la plataforma Arduino, el diseño de los componentes del guante y las técnicas de aprendizaje automático.

Posteriormente, se realizó el diseño del hardware, seleccionando los componentes electrónicos necesarios y definiendo su distribución sobre el guante. A continuación, se llevó a cabo el montaje del dispositivo, incluyendo la colocación de los sensores, el cableado y las conexiones necesarias para su funcionamiento. Una vez construido el guante, se inició la fase de adquisición de datos, en la que se recopiló un conjunto de datos propio a partir de la realización repetida de distintos signos considerados en el proyecto.

Finalmente, se desarrolló la fase de modelado, donde se entrenaron y validaron diferentes modelos de *machine learning* para el reconocimiento de signos.

3.2. Tecnologías

El desarrollo del proyecto se basa en la integración de diferentes tecnologías de hardware y software que permiten la adquisición de datos, el procesamiento de la información y la ejecución de modelos de *machine learning*.

El sistema se implementa sobre una placa Arduino Nano (ESP32) programada mediante Arduino IDE, la cual actúa como unidad central de procesamiento encargada de leer los sensores y ejecutar el modelo de clasificación entrenado previamente.

Para el diseño inicial del hardware se ha utilizado Tinkercad, una página web que permite realizar simulaciones de electrónica. Esta página se ha utilizado para aprender cómo se deben conectar los sensores de flexión, qué resistencias son las más óptimas y

cómo organizar los pines del arduino antes de implementarlo físicamente (ver Figura 3.1).

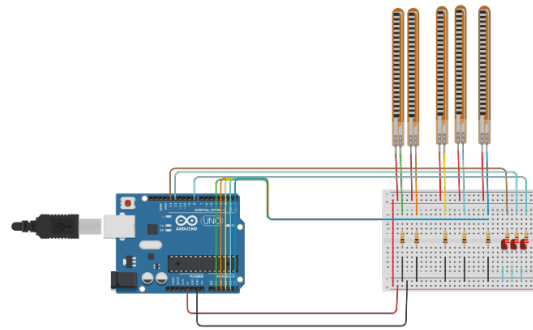


Figura 3.1: Ejemplo de circuito en Tinkercad.

Por otro lado, el desarrollo de software se ha realizado principalmente en Python; un lenguaje de programación versátil y fácil de aprender. Se utiliza habitualmente para análisis de datos, desarrollo web, automatización de tareas, inteligencia artificial, aprendizaje automático y creación de scripts o programas educativos. Además, gracias a las librerías externas de las que dispone, se hace más amena la tarea de realizar las conexiones con Arduino, así como montar el modelo de *machine learning* o *deep learning*. En este caso, se utilizará principalmente para entrenar el modelo de *machine learning* y para generar el conjunto de datos.

Asimismo, la programación del microcontrolador se ha llevado a cabo mediante Arduino IDE, una herramienta de programación que permite interactuar directamente con la placa. Esta herramienta puede integrar en la ESP32 las instrucciones que deben cumplirse y permite la adquisición de datos procedentes de los sensores del guante.

Finalmente, durante la fase de diseño electrónico, también se ha utilizado Kicad, un software dedicado al diseño de componentes electrónicos y placas PCB, que permite crear esquemas, placas y visualizar los resultados mediante un simulador en 3D.

3.3. Planificación del proyecto

En primer lugar, al no tener experiencia con Arduino ni electrónica, se utilizarán las primeras semanas para trabajar en la familiarización con estas tecnologías. Una vez introducido, se pasará a la investigación del estado del arte, que será necesaria para decidir los componentes que integran el guante y los modelos de *machine learning* que se utilizarán.

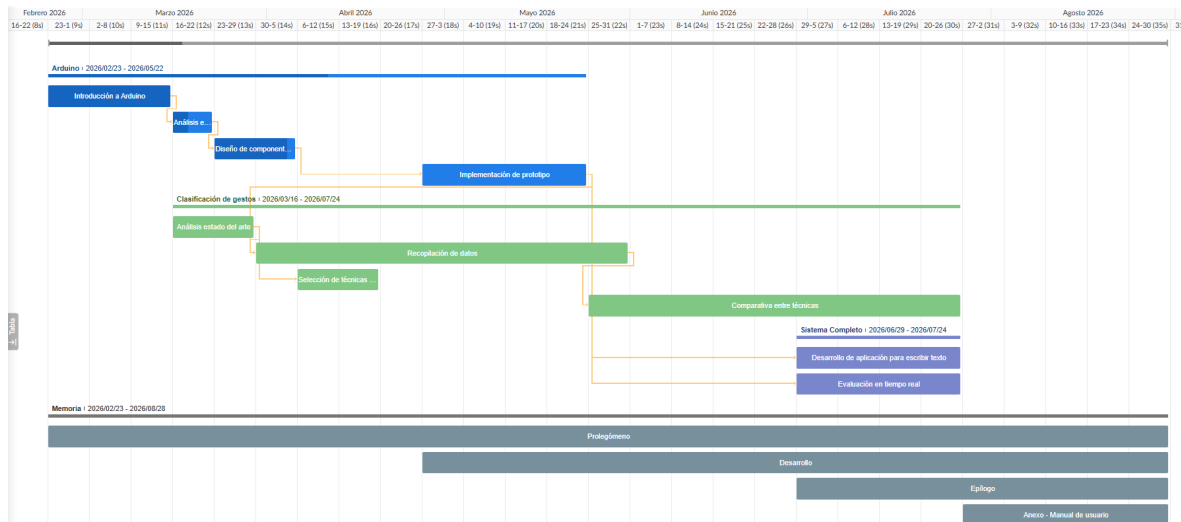


Figura 3.2: Diagrama de Gantt.

En el diagrama se puede apreciar que la recopilación de datos depende de la implementación del prototipo; esto se debe a la falta de conjuntos de datos relacionados con LSE, por lo que se usarán datos propios para crear uno. Esto será necesario para probar y comparar distintas técnicas de *machine learning*.

Por último, una vez completado el sistema, se pasará al desarrollo de una aplicación que recibirá las traducciones del guante en forma de texto y se probará en tiempo real.

En cuanto a la documentación del proyecto, esta siempre estará presente en todo momento, obteniendo más contenido con el desarrollo del guante.

3.4. Organización

Debido a la ausencia de bases de datos públicas de LSE, el desarrollo del sistema se ha estructurado alrededor de la creación de un conjunto de datos propio mediante el uso del guante.

La organización del proyecto sigue una construcción progresiva del sistema completo, desde el hardware hasta la capa de inteligencia artificial y la integración final.

En primer lugar, se realizó la adquisición de los materiales necesarios para la construcción del guante sensorizado. Posteriormente, se llevaron a cabo el estudio y las pruebas de los sensores, con el objetivo de comprender su comportamiento y realizar su calibración.

Una vez finalizada esta fase, se procedió al diseño e integración de los sensores de flexión en el guante. A continuación, se incorporó el giroscopio para la captura de los movimientos de la mano.

Posteriormente, se desarrolló un sistema de adquisición de datos destinado a la generación del conjunto de datos. Finalmente, se implementó el modelo de *machine*

learning encargado de asociar los signos realizados con las letras o palabras correspondientes.

3.5. Costes

A continuación, se detallarán los costes asociados al desarrollo del proyecto, diferenciando el coste material y el coste personal; este último calculado en función del tiempo dedicado al proyecto.

3.5.1. Coste material

Para la realización del proyecto, ha sido necesario adquirir diversos componentes electrónicos y materiales auxiliares para la construcción del guante y el desarrollo del sistema de adquisición de datos.

A continuación, se detalla el costo de los principales elementos utilizados:

- 8 sensores de flexión de 7 cm: 10.89€ por unidad.
- 4 sensores de flexión de 5 cm: 6.69€ por unidad.
- 2 sensores giroscópicos: 17.99€ por unidad.
- Guantes base: 12€.
- Resistencias: 3€.
- Cables de conexión: 4€.
- Cinta de doble cara: 6€.
- Protoboard (fase de pruebas): 3€.
- Placa ESP32: 18€.
- Soldador y material de soldadura: 30€.
- Placa de cobre y conectores: 13€.

Aunque en el guante no se han utilizado todos los elementos, se han tenido en cuenta los componentes de repuesto no utilizados.

3.5.2. Coste personal

A continuación, se muestra un cuadro que presenta tareas específicas y el tiempo total (en horas) dedicado a cada una de ellas. Algunos de los tiempos son estimaciones.

Tareas	Tiempo dedicado
Introducción a Arduino (pruebas de componentes)	50 horas
Análisis del estado del arte	50 horas
Montaje del guante	150 horas
Desarrollo del sistema de adquisición de datos	10 horas
Creación y preparación del conjunto de datos	50 horas
Entrenamiento del modelo de machine learning	15 horas
Documentación	150 horas
Total	478 horas

Tabla 3.1

Tiempo dedicado a cada tarea para la realización de este trabajo.

El costo total estimado del desarrollo del proyecto se calcula como:

$$\text{Coste total} = 478 \text{ horas} \times 20 \text{ €/hora} = 9560 \text{ €}.$$

Este valor representa una estimación del coste equivalente del desarrollo si se hubiera realizado en un entorno profesional (los perfiles junior suelen rondar los 20 €/h)

3.6. Riesgos

Durante el desarrollo del proyecto, se han identificado algunos riesgos que pueden afectar el funcionamiento del guante.

- Fragilidad de los componentes electrónicos: Los sensores de flexión, el giroscopio y la placa ESP32 son elementos sensibles que pueden dañarse durante el montaje o uso de los mismos.

Impacto: Alto. Probabilidad: Media-Alta.

- El sensor del dedo índice está un poco deteriorado y podría romperse. Esto se debe al proceso de soldadura del cable al propio sensor.

Impacto: Muy alto. Probabilidad: Muy baja.

- Si se toca el giroscopio durante el uso, este podrá dejar de proporcionar datos. Se puede solucionar pulsando el botón de reinicio de la placa arduino.

Impacto: Muy alto. Probabilidad: Alta.

- Durante la escritura del archivo CSV, este se puede corromper, dejando los datos de dicha etiqueta inutilizables.

Impacto: Muy alto. Probabilidad: Baja.

3.7. Gestión de la configuración

Para manejar las versiones en el desarrollo del código del guante, se utilizará un repositorio de GitHub, siendo la nomenclatura la siguiente: dpc vA.B.C, donde A,B y C son números.

- A: Cambio o avance muy importante (creación de nuevas funciones, grandes avances,...).
- B: Cambio o avance significativo (terminar funciones ya creadas, solucionar problemas,...).
- C: Cambio de poca importancia (nombres de variables, comentarios, estética,...).

Parte II

Desarrollo

4. Análisis del sistema

4.1. Requisitos del sistema

4.1.1. Requisitos funcionales

El sistema debe ser capaz de capturar, procesar y clasificar los signos realizados por el usuario mediante el guante sensorizado, transformándolos en una representación textual.

Los requisitos funcionales son los siguientes:

- RF1 - El sistema debe ser capaz de recibir los datos procedentes de los sensores de flexión y de la unidad de medida inercial (IMU) del guante.
- RF2 - El sistema debe permitir la adquisición y almacenamiento de datos en formato CSV para la creación del conjunto de datos de entrenamiento.
- RF3 - El sistema debe procesar los datos obtenidos para su uso en un modelo de *machine learning*, aplicando técnicas de preprocesamiento y extracción de características.
- RF4 - El sistema debe clasificar los signos realizados por el usuario en las clases definidas durante el entrenamiento del modelo.
- RF5 - El sistema debe generar una salida textual correspondiente al signo realizado.

4.1.2. Requisitos no funcionales

Adecuación funcional - RNF1

El sistema debe proporcionar las funcionalidades necesarias para cumplir con los objetivos establecidos, garantizando la correcta captura, procesamiento y clasificación de los signos realizados con el guante sensorizado.

En este sentido, se asegura la completitud funcional mediante la inclusión de todas las etapas del proceso, desde la adquisición de datos procedentes de los sensores, su preprocesamiento y transformación en vectores de características, hasta la inferencia mediante un modelo de machine learning.

Asimismo, se garantiza la corrección funcional, de forma que los resultados obtenidos sean coherentes con los datos de entrada y con el comportamiento esperado del sistema, reduciendo posibles errores en la clasificación de los signos.

Por último, la pertinencia funcional se asegura mediante la implementación exclusiva de funcionalidades orientadas al objetivo del sistema, evitando procesos redundantes o ajenos al problema de la traducción de signos.

Eficiencia de desempeño - RNF2

El sistema debe garantizar un rendimiento adecuado durante la ejecución en tiempo real, de forma que el procesamiento de los datos procedentes del guante y la obtención de predicciones no introduzca retardos perceptibles en la interacción con el usuario.

En este sentido, se tiene en cuenta el comportamiento temporal del sistema, asegurando que las operaciones de adquisición de datos, preprocesamiento y clasificación se realicen dentro de intervalos compatibles con el funcionamiento en tiempo real.

Asimismo, se optimiza la utilización de recursos mediante el uso de estructuras de datos eficientes y la ejecución de tareas en hilos independientes, evitando el bloqueo de la interfaz gráfica durante los procesos de lectura, entrenamiento e inferencia.

Por último, se considera la capacidad del sistema para mantener un rendimiento estable durante la ejecución continuada, especialmente durante la captura de datos en ventanas temporales y la inferencia continua del modelo.

Compatibilidad - RNF3

El sistema presenta compatibilidad con los entornos necesarios para su ejecución, basados en Python y sus librerías asociadas, lo que permite su despliegue en diferentes sistemas operativos sin modificaciones significativas.

Asimismo, mantiene compatibilidad con el microcontrolador Arduino utilizado para la adquisición de datos, estableciendo comunicación mediante puerto serie o conexión WiFi, lo que garantiza la correcta transferencia de información entre el hardware y el software.

Fiabilidad - RNF4

El sistema debe garantizar un funcionamiento fiable durante la adquisición, procesamiento y clasificación de los datos procedentes del guante sensorizado.

En este sentido, debe ser capaz de gestionar posibles errores derivados de la comunicación con el microcontrolador, ya sea mediante conexión serie o WiFi, evitando que estos provoquen la interrupción completa de la aplicación.

Asimismo, se tiene en cuenta la presencia de datos erróneos debido a fallos de hardware, por lo que estos no terminarán con la ejecución del programa, sino que el mismo mostrará en pantalla que se está produciendo un error.

Además, el sistema debe mantener un comportamiento estable durante la ejecución prolongada de la inferencia en tiempo real, garantizando la continuidad del proceso de clasificación sin bloqueos ni pérdidas significativas de datos.

Capacidad de interacción - RNF5

El sistema debe proporcionar una interacción adecuada con el usuario, facilitando su uso mediante una interfaz gráfica que permita controlar de forma sencilla los distintos modos de funcionamiento.

En este sentido, la aplicación debe ser operable de manera intuitiva, permitiendo iniciar, pausar y detener los procesos de captura de datos y de inferencia en tiempo real sin necesidad de conocimientos técnicos avanzados.

Asimismo, el sistema debe proporcionar una retroalimentación clara al usuario, mostrando el estado de la adquisición de datos, el progreso del proceso de lectura y los resultados de la clasificación de los signos.

Además, se considera la protección frente a errores de usuario, evitando que acciones incorrectas provoquen fallos en la ejecución del sistema mediante el control del flujo de la aplicación y la gestión de estados internos.

Por último, se garantiza que el sistema sea accesible en su uso básico, proporcionando una estructura de interacción coherente entre las distintas ventanas y funcionalidades de la aplicación.

4.1.3. Requisitos de información

Para que el sistema funcione, se debe entrenar un modelo de *machine learning*, por lo que es necesario disponer de una base de datos extensa. Sin embargo, también es importante entender cuáles son los datos con los que se va a trabajar:

- RI1 - Datos de los sensores de flexión: Se registran los voltajes generados por los sensores flexibles al doblarse cada dedo, leídos por los pines analógicos del microcontrolador.
 - Unidad de medida: unidades de lectura ADC (0–4095).
 - El microcontrolador transmite los datos directamente sin aplicar ningún tipo de normalización, ya que esta fase se realiza posteriormente en el módulo de procesamiento del sistema.
- RI2 - Datos del giroscopio: Se obtienen a partir del sensor inercial MPU6050, que proporciona tanto la aceleración lineal en los tres ejes como la velocidad angular del dispositivo.
 - Acelerómetro: mide la aceleración en los tres ejes, permitiendo estimar si la mano está o no en movimiento.

- Giroscopio: mide la velocidad angular en los tres ejes, expresada en grados por segundo, lo que permite capturar los movimientos de rotación de la mano.

4.2. Casos de uso

4.2.1. Modelos de casos de uso

En esta sección se presenta el modelo de casos de uso del sistema desarrollado. Este modelo tiene como objetivo representar de forma estructurada las interacciones entre el usuario y el sistema, identificando los distintos escenarios de funcionamiento y el comportamiento esperado en cada uno de ellos.

Para ello, se han definido los actores implicados y los casos de uso principales del sistema, los cuales se describen mediante su flujo de ejecución, así como sus precondiciones, postcondiciones y posibles escenarios alternativos. A continuación, se presenta el diagrama de casos de uso del sistema (Figura 4.1).

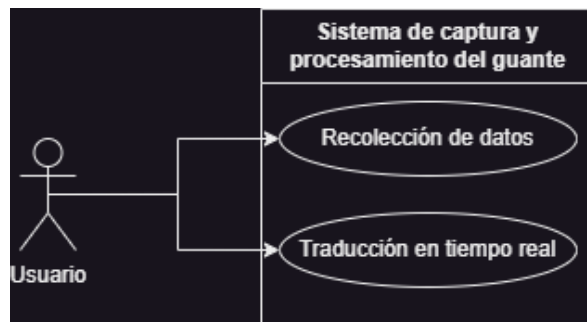


Figura 4.1: Diagrama de casos de usos.

Caso de uso: Recolección de datos

Actores del sistema:

- Usuario: Persona que utiliza el guante.

Precondiciones:

- Todos los sensores del guante están conectados y en funcionamiento.
- El usuario tiene el guante bien colocado.
- La placa Arduino debe estar conectada directamente al equipo externo mediante un puerto USB.
- El sistema de recepción y procesamiento del equipo externo está en funcionamiento.

Postcondiciones:

- Los datos de los sensores quedan almacenados en archivos CSV etiquetados.
- Los datos quedan disponibles para el entrenamiento del modelo.

Flujo principal:

1. El usuario se coloca el guante.
2. El usuario inicia el sistema para la adquisición de datos de los sensores de flexión y de la IMU.
3. El Arduino envía los datos al equipo externo mediante el puerto serie.
4. El equipo recibe los datos y los procesa.
5. El software del equipo externo almacena las muestras en archivos CSV, etiquetándolas según la clase correspondiente.
6. Los distintos archivos CSV generados se unifican en un único conjunto de datos utilizando el *script UnirDatos*.

Flujo alternativo:

- A1. Fallo de comunicación. Si se pierde la comunicación entre el Arduino y el equipo externo, la adquisición de datos se detiene hasta que se recupere la conexión.

- A2. Fallo de conexión de los sensores. Si los sensores no devuelven valores válidos, significa que ha habido un problema con los cables que los conectan a la placa del guante.

Caso de uso: Traducción en tiempo real

Actores del sistema:

- Usuario: Persona que utiliza el guante.

Precondiciones:

- El sistema dispone de un conjunto de datos almacenados en formato csv.
- El sistema está inicializado correctamente.
- El guante está bien colocado en la mano del usuario.
- La conexión entre el arduino y el equipo externo está establecida (USB o Wifi).

Postcondiciones

- El sistema devuelve la predicción del signo realizado, junto con un porcentaje de confianza.
- El resultado se muestra en la pantalla del equipo externo.

Flujo principal:

1. El usuario realiza el signo con la mano.
2. El Arduino recibe los valores de los sensores.
3. El equipo externo recibe y procesa los datos del Arduino.
4. Se extraen las características necesarias para la predicción.
5. El modelo de clasificación procesa los datos y realiza la predicción.
6. El equipo muestra en pantalla los tres mejores candidatos y el seleccionado entre ellos.

Flujo alternativo:

- A1. Fallo de comunicación. Si se pierde la comunicación entre el Arduino y el equipo externo, la adquisición de datos se detiene hasta recuperar la conexión.

- A2. Modelo no disponible. Si no hay una base de datos generada, no se podrá entrenar el modelo de aprendizaje y, por lo tanto, no habrá traducción en tiempo real.

4.2.2. Modelo conceptual

En esta sección se presentará el modelo conceptual del sistema desarrollado. Se identifican las principales clases del sistema, así como sus relaciones, con el objetivo de representar la estructura del proyecto mediante un diagrama de clases UML (ver Figura 4.2).

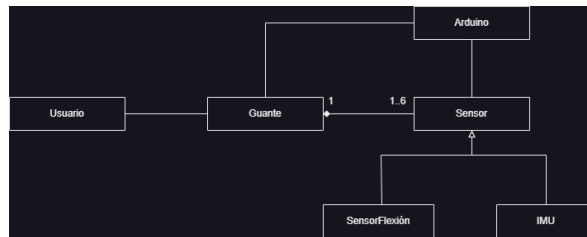


Figura 4.2: Diagrama UML.

En este modelo se incluyen el usuario, el guante, los sensores y el microcontrolador Arduino. El guante actúa como elemento central del sistema, integrando los sensores de flexión y la unidad de medida inercial (IMU), mientras que el Arduino se encarga de la adquisición y transmisión de los datos hacia el sistema de procesamiento. Además, los sensores se especializan en dos tipos: Aquellos encargados de la detección de la flexión de los dedos y aquellos responsables de la medición de la orientación y la aceleración de la mano.

4.2.3. Modelo de interfaz de usuario

En esta sección se presenta la interfaz de usuario desarrollada para facilitar la utilización del guante instrumentado. La interfaz permite al usuario recolectar datos para la creación de conjuntos de entrenamiento y realizar la traducción de signos en tiempo real.

Con el objetivo de simplificar la interacción con el sistema, la aplicación ha sido diseñada centralizando en una única ventana las opciones de configuración y control correspondientes a cada modo de funcionamiento. Este enfoque reduce la complejidad de uso y permite una transición más directa entre las distintas funciones del sistema.

Asimismo, se presenta un diagrama de navegación que representa la estructura de pantallas disponibles y la navegación entre estas (ver Figura 4.3).

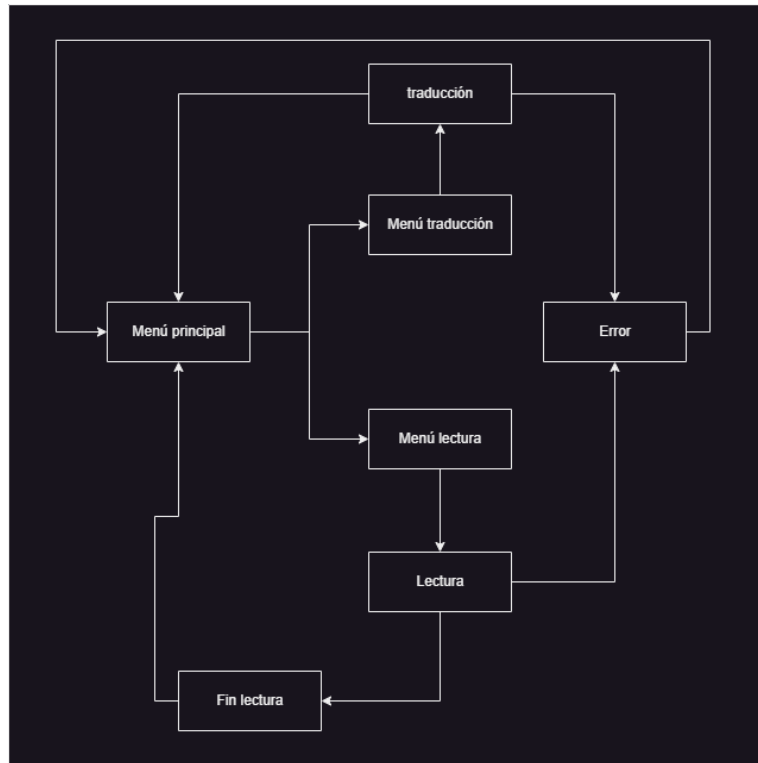


Figura 4.3: Diagrama de navegación.

5. Diseño del sistema

En este capítulo se recoge el diseño de la arquitectura del sistema informático, la parametrización del software base (opcional), el diseño físico de datos y el diseño detallado de la interfaz de usuario.

5.1. Arquitectura del sistema

En esta sección se describe la arquitectura general del sistema desde el punto de vista lógico, definiendo la estructura de los distintos componentes de software que lo conforman y las relaciones existentes entre ellos. El objetivo es presentar una visión global de cómo se organiza el sistema internamente para dar soporte a la adquisición, procesamiento y clasificación de los datos procedentes del guante instrumentado.

5.1.1. Diseño de alto nivel

Siguiendo la aproximación del modelo C4, en este apartado se presenta una visión general de la arquitectura del sistema, cuyo objetivo principal es el reconocimiento de signos mediante un guante sensorizado. Esta aplicación de escritorio permite tanto la adquisición de datos en bruto procedentes de los sensores como la clasificación de signos en tiempo real, empleando un modelo de *machine learning* previamente entrenado.

Desde una perspectiva de contexto, el sistema cuenta con dos actores principales. Por un lado, el usuario final, quien se coloca el guante e interactúa con la interfaz gráfica para recolectar datos o iniciar la traducción en tiempo real; y, por otro lado, el guante sensorizado, que incorpora un microcontrolador Arduino, actúa como puente de comunicación, encargándose de la captura y el envío de los datos de los sensores.

A un nivel de contenedores, el sistema se estructura en tres grandes bloques funcionales que operan de manera coordinada.

1. La interfaz de usuario, implementada mediante la librería *CustomTkinter*, proporciona los elementos visuales necesarios para la navegación entre los distintos modos de funcionamiento (la recolección de datos y la traducción de signos).
2. Un componente de preprocesamiento y clasificación, encargado de extraer características de los datos, entrenar el modelo de *machine learning* y realizar la inferencia en tiempo real para la identificación de signos.
3. Un módulo de comunicación con el microcontrolador, responsable de gestionar la conexión con el dispositivo Arduino y permitir la adquisición de datos provenientes del guante sensorizado.

A continuación, se presentan los diagramas de contexto y de contenedores correspondientes al modelo C4, que permiten visualizar la arquitectura del sistema desde

distintos niveles de abstracción. El diagrama de contexto (ver Figura 5.1) muestra la interacción entre el usuario, el guante sensorizado y el sistema, mientras que el de contenedores (ver Figura 5.2) desglosa este último en sus tres bloques funcionales principales.

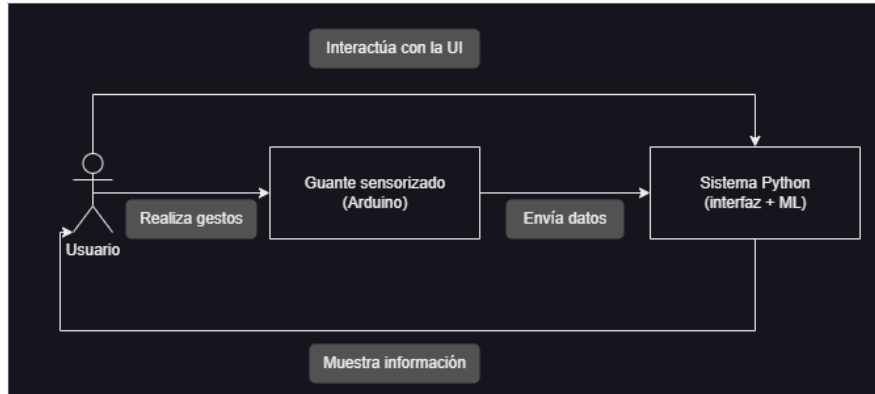


Figura 5.1: Diagrama de contexto.

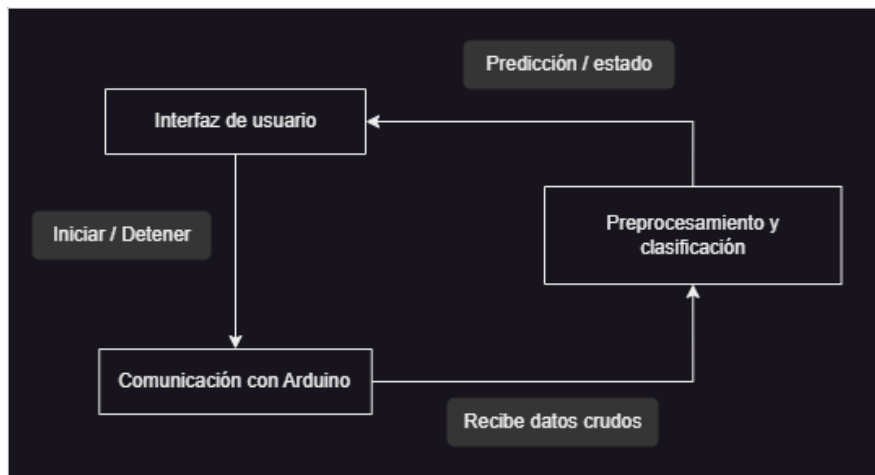


Figura 5.2: Diagrama de contenedores.

5.1.2. Diseño detallado

En esta sección se describe la estructura interna de los principales módulos encargados del procesamiento de datos, la comunicación con el microcontrolador y la ejecución del modelo de *machine learning*, mostrando la forma en que interactúan entre sí. El objetivo es presentar cómo se organiza el código fuente para dar soporte a las distintas funcionalidades de la aplicación, incluyendo la adquisición de datos, su procesamiento y la inferencia en tiempo real. La Figura 5.3 muestra la arquitectura modular del sistema y las relaciones entre los distintos componentes.

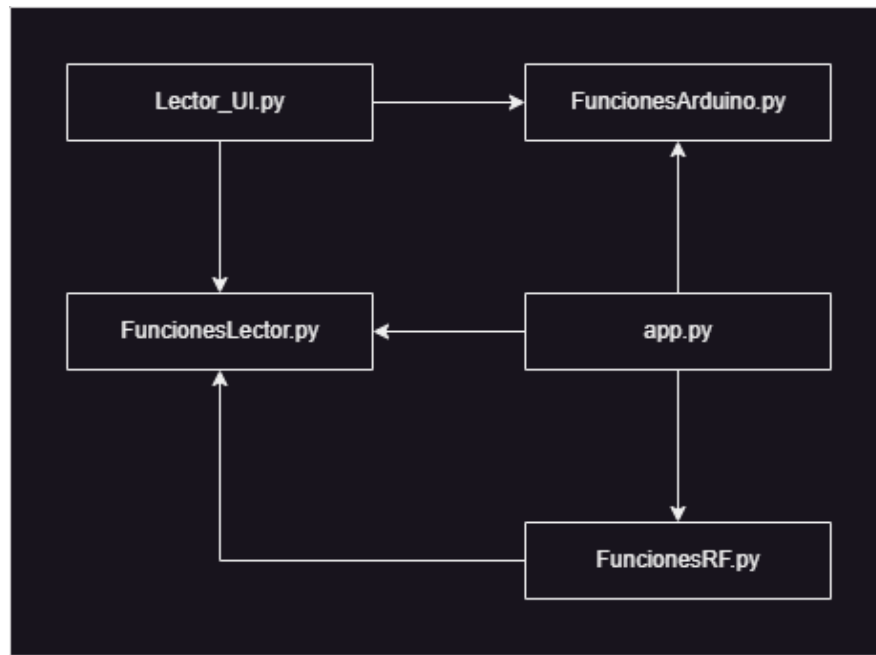


Figura 5.3: Diagrama de componentes.

La implementación del sistema se organiza en distintos módulos, cada uno de ellos especializado en una funcionalidad concreta. La Figura 5.3 refleja las relaciones entre dichos módulos, mientras que la estructura general del proyecto se muestra a continuación.

```
| /TFG
| - app.py
| - Launcher.py
| - FuncionesArduino.py
| - FuncionesLector.py
| - FuncionesRF.py
| - Lector_UI.py
| - Traducccion_UI.py
| - UnirDatos.py
| - /Dataset
|   | - a.csv
|   | - b.csv
|   ...
```

El código fuente completo del proyecto se encuentra disponible en el repositorio de GitHub:

<https://github.com/percorb/TFG>

En dicho repositorio se puede consultar la implementación completa de los módulos descritos en esta sección, así como la organización del proyecto y el historial de cambios.

FuncionesLector.py

El módulo *FuncionesLector.py* constituye el puente entre la adquisición de datos y el modelo de *machine learning*, transformando las señales en bruto procedentes del guante en un conjunto de características numéricas adecuadas para su clasificación [12].

En primer lugar, se calculan medidas estadísticas sobre las señales de flexión de cada dedo, incluyendo la media y la desviación estándar dentro de cada ventana de muestreo. Estas métricas permiten capturar tanto el valor medio del signo como su variabilidad temporal.

A partir de los datos del acelerómetro, se calculan las variables de orientación *pitch* y *roll*, que describen la inclinación de la mano en el espacio. Estas variables se resumen mediante la mediana y su rango de variación dentro de cada ventana, con el objetivo de capturar la dinámica del movimiento.

Además, se calculan magnitudes vectoriales tanto del acelerómetro como del giroscopio, obtenidas como la norma euclidiana de sus componentes en los tres ejes. A partir de estas magnitudes se extraen la media y la desviación estándar, proporcionando información global sobre la intensidad del movimiento.

Por último, el módulo incorpora un mecanismo de validación que descarta ventanas de datos no válidas durante el proceso de recolección.

FuncionesRF.py

El módulo *FuncionesRF.py* implementa la lógica relacionada con el modelo de *machine learning*, incluyendo tanto el entrenamiento del clasificador como la inferencia en tiempo real a partir de las características extraídas del sistema de adquisición.

Durante la fase de entrenamiento, el módulo realiza la carga del conjunto de datos almacenado en formato CSV, la separación entre las variables de entrada y las etiquetas correspondientes a cada signo, así como el entrenamiento del modelo utilizando las características previamente procesadas por el módulo *FuncionesLector.py*. El modelo utilizado es un clasificador basado en *Random Forest*, encargado de realizar la clasificación de los signos a partir de los vectores de características obtenidos durante el preprocesamiento.

En cuanto al proceso de inferencia en tiempo real, el sistema recibe vectores de características ya procesados y realiza la predicción del signo correspondiente mediante el modelo entrenado. Para mejorar la estabilidad de la salida, se implementa un sistema

de suavizado basado en el historial de predicciones recientes (hasta siete elementos), de forma que la predicción final corresponda al valor más frecuente dentro de una ventana de resultados. El modelo no solo devuelve la clase más probable, sino también las tres clases con mayor probabilidad, proporcionando información adicional sobre la confianza de la predicción.

Asimismo, el proceso de traducción se ejecuta en segundo plano mediante el uso de hilos, permitiendo mantener la interfaz gráfica operativa durante la clasificación y facilitando la detención segura del proceso cuando el usuario lo solicita.

FuncionesArduino.py

El módulo *FuncionesArduino.py* se encarga de gestionar la comunicación con el microcontrolador Arduino, proporcionando los mecanismos necesarios para establecer la conexión entre el sistema software y el dispositivo hardware. Este módulo actúa como punto de acceso para la adquisición de datos procedentes del guante sensorizado, abstrauyendo el mecanismo de comunicación utilizado y permitiendo al resto de componentes acceder a la información de los sensores de forma transparente.

En concreto, implementa dos métodos de conexión: una conexión mediante puerto serie (USB), en la que se establece comunicación directa con el dispositivo a través del puerto COM correspondiente, y una conexión mediante WiFi basada en sockets TCP, que permite la transmisión de datos a través de la red local. Para el caso de la conexión WiFi, el módulo incorpora una implementación asíncrona basada en hilos con el objetivo de evitar el bloqueo de la interfaz gráfica durante el establecimiento de la conexión. Asimismo, dispone de un mecanismo de cancelación que permite interrumpir el proceso cuando el usuario lo desee.

Firmware del microcontrolador

El firmware del microcontrolador es el encargado de adquirir los datos procedentes de los sensores del guante y transmitirlos al equipo donde se ejecuta la aplicación principal. Durante la inicialización se configura la comunicación serie y el bus I2C, utilizado para establecer la comunicación con el sensor MPU6050. En cada iteración del programa, se realiza la lectura de los cinco sensores de flexión mediante las entradas analógicas del microcontrolador y, posteriormente, se obtiene la información del acelerómetro y del giroscopio a través del protocolo I2C.

Una vez adquiridos los datos, los valores del acelerómetro y del giroscopio se convierten a sus correspondientes unidades físicas utilizando los factores de escala definidos para la configuración empleada. Finalmente, toda la información capturada se envía en formato CSV.

5.2. Parametrización del software base

El sistema ha sido desarrollado en un entorno basado en Python 3.14 sobre el sistema operativo Windows. Para el desarrollo de la aplicación, se ha empleado Visual Studio Code como entorno de desarrollo, mientras que el firmware del microcontrolador Arduino se ha implementado mediante el Arduino IDE utilizando el lenguaje C/C++ propio de la plataforma.

Para su correcto funcionamiento, ha sido necesario configurar el entorno de ejecución mediante la instalación de diversas bibliotecas externas, entre las que se incluyen *NumPy*, *Pandas* y *Scikit-learn*, así como módulos estándar de Python como *os* o *threading*. Estas dependencias proporcionan las funcionalidades necesarias para el procesamiento de datos, el entrenamiento del modelo de *machine learning* y la gestión de la concurrencia.

El control de versiones se ha realizado mediante Git, permitiendo mantener un seguimiento de los cambios realizados durante el desarrollo y facilitando la organización del código fuente.

En relación con la comunicación con el microcontrolador, el sistema permite la conexión tanto mediante puerto USB como a través de la conexión WiFi, en función de la disponibilidad del dispositivo. En el caso de la comunicación serie, el puerto COM puede variar, por lo que debe configurarse adecuadamente antes de la ejecución del sistema.

Asimismo, el sistema requiere que la estructura de directorios del proyecto se mantenga de forma que los archivos del conjunto de datos puedan localizarse correctamente durante el proceso de entrenamiento y clasificación.

5.3. Diseño físico de datos

El sistema no utiliza un sistema de bases de datos relacionales, sino que la persistencia de los datos se realiza mediante archivos en formato CSV. Durante el proceso de adquisición, cada signo se almacena inicialmente en un archivo CSV independiente. Posteriormente, estos archivos pueden combinarse para formar un conjunto de datos global que se emplea en el entrenamiento del modelo de machine learning. Estos archivos contienen las muestras recogidas durante el proceso de adquisición de datos del guante sensorizado.

Cada fichero CSV corresponde a un conjunto de datos asociado a una clase o signo concreto, donde cada fila representa una muestra y cada columna, una característica derivada de los sensores. Entre estas características se incluyen valores procedentes de los sensores de flexión, así como variables calculadas a partir del acelerómetro y el giroscopio.

Los datos son procesados mediante librerías como *Pandas* y *NumPy*, lo que permite su utilización directa en el entrenamiento de modelos de *machine learning*. La

estructura concreta de cada fichero CSV se compone de 19 atributos, que se dividen en la etiqueta de clase y las características extraídas de los sensores del guante.

La primera columna corresponde a la etiqueta, que identifica el signo asociado a cada muestra. A continuación, se incluyen los valores de flexión de cada dedo. Dado que cada muestra en el conjunto de datos es el resultado de procesar una ventana de diez muestras, se almacenan la media y la desviación estándar de la flexión de cada dedo. Esto último permite capturar la variabilidad del movimiento durante la ventana de muestreo.

Posteriormente, se incorporan características del sensor inercial, como la mediana del *pitch* y el *roll*, así como su rango de variación, que aporta información sobre la dinámica del movimiento.

El *pitch* y el *roll* son medidas que representan la orientación de la mano en el espacio. El *pitch* describe la inclinación hacia adelante o hacia atrás, mientras que el *roll* indica la inclinación lateral de la mano. Estas variables se calculan a partir de los componentes del acelerómetro mediante el uso de la función *arctan2*, lo que permite obtener una estimación robusta de la orientación.

Finalmente, se incluyen medidas estadísticas de las magnitudes vectoriales, tanto del acelerómetro como del giroscopio, concretamente la media y la desviación estándar de dichas magnitudes, que resumen el comportamiento global del movimiento en los tres ejes.

El conjunto de datos final está formado por 5860 muestras, distribuidas entre 31 clases correspondientes a las letras del alfabeto LSE y a varios signos asociados a palabras completas. Cada instancia contiene un total de 19 columnas, correspondientes a la etiqueta de clase y a las 18 características extraídas de los sensores del guante.

6. Montaje del guante

6.1. Descripción general

El sistema de captura de movimiento se basa en un guante instrumentado que traduce los movimientos de la mano en señales eléctricas. Para ello se han integrado cinco sensores de flexión analógicos (uno para cada dedo) [13] y una Unidad de Medición Inercial (IMU) MPU6050 [14], encargada de registrar la orientación, inclinación y giros de la mano en los tres ejes del espacio [15].

Toda la electrónica de acondicionamiento de señal, las conexiones y el microcontrolador Arduino Nano se centralizan en una placa de PCB acoplada al propio guante. De este modo, se consigue un dispositivo compacto y ergonómico que permite al usuario realizar los movimientos de la lengua de signos de manera cómoda y natural, enviando los datos para su posterior traducción.

6.2. Componentes utilizados

Para la construcción del prototipo físico del guante traductor, se han seleccionado una serie de componentes atendiendo a criterios de tamaño, coste y funcionalidad. A continuación, se detallan los elementos principales y el motivo de su selección:

- Guantes de deporte: Se seleccionó un modelo elástico y transpirable. Este diseño permite que el guante se ajuste firmemente a la mano del usuario, garantizando que los sensores no se desplacen demasiado de su posición original durante los movimientos.
- Sensores de flexión de 7 cm y 5 cm: Estos sensores varían su resistencia eléctrica de forma proporcional a su ángulo de curvatura. Se han utilizado cuatro sensores de 7 cm para los dedos índice, corazón, anular y meñique debido a su longitud. Para el dedo pulgar, dado que tiene una longitud menor, se ha utilizado el sensor de 5 cm. Se podría haber utilizado el sensor de 5 cm para el meñique, pero este no llegaba a cubrir el espacio requerido por muy poco.
- Arduino Nano ESP32: Se seleccionó esta placa de desarrollo porque combina el factor compacto de la familia Nano con la potencia de procesamiento y memoria del chip ESP32. Cuenta con conectividad nativa WiFi y Bluetooth, lo que dota al prototipo de capacidades inalámbricas para un dispositivo móvil y ofrece los pines analógicos necesarios para poder utilizar los sensores de flexión y el MPU6050.
- Placa de cobre PCB y conectores: La placa de cobre ha permitido desarrollar un circuito impreso a medida para centralizar todas las conexiones de forma limpia y segura.

6.3. Conexiones eléctricas y esquema del circuito

Una vez seleccionados los componentes del sistema, fue necesario diseñar el circuito encargado de concentrar todos los elementos en el menor espacio posible para evitar la reducción de la mano al colocar una placa en el guante.

La Figura 6.1 muestra el esquema eléctrico completo desarrollado para el prototipo mediante el uso de la herramienta KiCad. En él se pueden observar las conexiones de los sensores al microcontrolador, así como las resistencias.

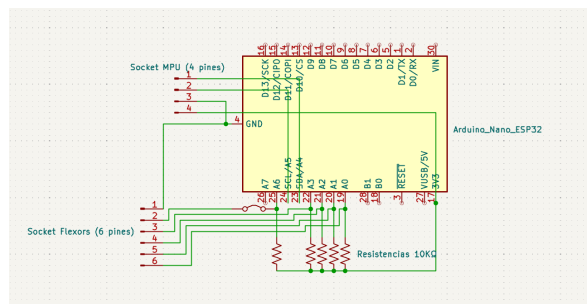


Figura 6.1: Esquema eléctrico del sistema desarrollado en KiCad.

Los sensores de flexión se conectan mediante divisores de tensión formados por una resistencia fija (10 K Ω) y el propio sensor. De esta forma, las variaciones de resistencia producidas por la curvatura de los dedos se transforman en variaciones de tensión que pueden ser medidas por las entradas analógicas del microcontrolador.

Por su parte, el módulo MPU6050 se conecta directamente al microcontrolador a través del bus I2C, utilizando las líneas SDA y SCL para la comunicación de datos.

6.3.1. Diseño de la PCB

A partir del esquema eléctrico descrito anteriormente, se desarrolló una placa de circuito impreso con el objetivo de integrar todos los componentes del sistema en un único soporte físico. Esta decisión permite reducir el número de cables sueltos, mejorar la fiabilidad del sistema y facilitar su integración en el guante, manteniendo la ergonomía del dispositivo. El diseño de la PCB se realizó utilizando la herramienta KiCad, a partir directamente del esquema eléctrico.

En primer lugar, se organizaron los componentes de forma que ocupen el menor espacio posible en la placa, logrando que adopte la dimensión de $3cm \times 5,5cm$, siendo perfecta para no sobresalir del guante.

Posteriormente, se realizó el ruteado de las pistas de cobre, conectando todas las señales de acuerdo con el esquema eléctrico y buscando la mejor forma de aprovechar el poco espacio disponible.

La Figura 6.2 muestra el diseño final de la placa de circuito impreso, donde se puede observar la disposición de los componentes y las rutas de las conexiones.

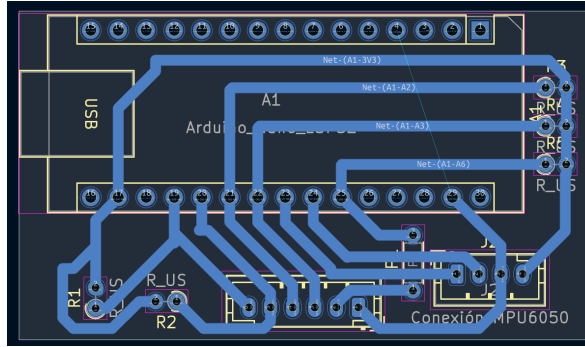


Figura 6.2: Diseño de PCB desarrollado en KiCad.

6.4. Fabricación y montaje del sistema

A continuación, se describe el proceso de fabricación del sistema, desde la obtención de la placa PCB hasta la integración de todos los componentes en el guante.

6.4.1. Fabricación de la PCB

Una vez finalizado el diseño de la placa de circuito impreso, se procedió a su fabricación. La PCB fue generada a partir del diseño, aunque no sea completamente fiel debido a las diferencias de los elementos que proporciona KiCad para el diseño y los recursos disponibles para hacerlo realidad.

Todos los componentes (resistencias, conectores y sócalos) han sido soldados a la placa. Sin embargo, para evitar soldar el microcontrolador en la placa, se han soldado dos sócalos, permitiendo que el Arduino pueda ser extraído cuando sea necesario. Además, esto permite que debajo del Arduino haya espacio para colocar algún componente.

Posteriormente, se ha seguido el diseño de PCB para formar las rutas que permitirán la conexión entre los componentes. Para ello, se han utilizado patillas de resistencias como puentes para el proceso de soldadura, de forma que se pueda crear un camino con estaño fundido.

En cuanto a la conexión de los sensores al microcontrolador, se han utilizado bornas de conexión, las cuales garantizan un contacto estable y facilitan su manipulación, permitiendo, además, su desconexión en caso de mantenimiento o sustitución.

De este modo, se ha obtenido una placa compacta y funcional que integra todos los elementos electrónicos y actúa como núcleo de interconexión del guante.

6.4.2. Integración de los componentes en el guante

La integración de los componentes electrónicos en el guante se ha realizado con el objetivo de garantizar una sujeción estable sin comprometer la comodidad ni la movilidad de la mano durante su uso.

La placa PCB se ha colocado en la parte superior del guante, más concretamente en la zona de la muñeca, ya que esta región proporciona una superficie relativamente estable y no interfiere con el movimiento de los dedos. Su fijación se ha realizado mediante una combinación de cinta de doble cara y costuras, lo que permite asegurar una sujeción firme y evitar desplazamientos durante el uso.

Por otro lado, los sensores de flexión se han dispuesto a lo largo de cada uno de los dedos, asegurando su correcta alineación con las articulaciones correspondientes para garantizar mediciones coherentes durante la flexión. Su fijación también se ha realizado mediante una combinación de cinta de doble cara y costuras, lo que proporciona una sujeción resistente y evita desplazamientos durante el movimiento repetitivo de la mano.

En cuanto a la alimentación del microcontrolador, esta depende del modo de funcionamiento del sistema. Si la conexión se realiza mediante USB, la placa se alimenta directamente del equipo al que está conectada. En cambio, cuando la comunicación se realiza mediante WiFi, se emplea una batería externa para suministrar la alimentación necesaria. Esta batería proporciona una tensión de 5V, equivalente a la suministrada por el puerto USB del equipo. Esta batería se guardará en un brazalete deportivo para permitir al usuario libertad de movimiento.

Todos los elementos del sistema se han distribuido de forma que el cableado no interfiera con el movimiento natural del usuario, agrupando las conexiones hacia la zona de la PCB para su posterior interconexión (ver Figura 6.3).

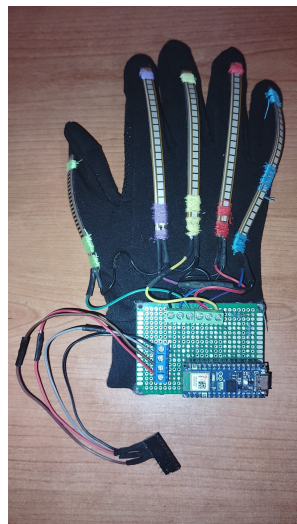


Figura 6.3: Prototipo del guante.

7. Aprendizaje de gestos

7.1. Objetivo de la comparación

La selección del algoritmo de aprendizaje automático constituye una de las decisiones más importantes en el desarrollo del sistema, ya que influye directamente en la precisión de las predicciones, el tiempo necesario para entrenar el modelo y el rendimiento durante la fase de inferencia. Por este motivo, antes de seleccionar el modelo definitivo, se realizó un breve estudio comparativo entre diferentes algoritmos de clasificación pertenecientes al ámbito del *machine learning*.

El objetivo de esta comparación es identificar el modelo que ofrece el mejor equilibrio entre capacidad de clasificación y eficiencia computacional para el conjunto de datos empleado en este trabajo. Para ello, todos los algoritmos fueron optimizados previamente mediante un proceso de búsqueda de hiperparámetros y evaluados bajo las mismas condiciones experimentales, garantizando así una comparación objetiva y reproducible.

Con el fin de obtener una visión completa del comportamiento de cada modelo, se analizaron diferentes métricas de rendimiento, entre las que se incluyen la exactitud (*Accuracy*), la precisión (*Precision*), la sensibilidad (*Recall*), la medida F1 (*F1-score*), así como los tiempos de entrenamiento y de predicción. A partir de los resultados obtenidos, se seleccionó el algoritmo que mejor se adapta a los requisitos del sistema desarrollado.

7.2. Metodología de evaluación

Con el propósito de realizar una comparación objetiva entre los diferentes algoritmos de clasificación, se definió una metodología de evaluación común para todos los modelos considerados. De este modo, todos los algoritmos fueron sometidos al mismo procedimiento de optimización, entrenamiento y evaluación, garantizando que las diferencias observadas en los resultados se debieran exclusivamente a las características de cada modelo y no a variaciones en el proceso experimental.

En los siguientes apartados se describen los algoritmos evaluados, el procedimiento empleado para la optimización de sus hiperparámetros, la estrategia de validación utilizada y las métricas consideradas para analizar su rendimiento.

7.2.1. Algoritmos de clasificación

Durante la revisión bibliográfica, se identificaron diferentes enfoques para el reconocimiento de signos mediante guantes sensorizados, incluyendo algoritmos de *machine learning*, técnicas de *deep learning* y métodos basados en reglas. A continuación, se describen brevemente los principales algoritmos encontrados en los trabajos analizados.

Random Forest es un algoritmo basado en un conjunto de árboles de decisión¹ [16] que mejora la estabilidad del modelo al reducir la varianza. Cada árbol se entrena con una muestra aleatoria del conjunto de datos y la clasificación final se obtiene combinando las predicciones de todos ellos. Este enfoque se emplea en proyectos como *Smart-glove* y el proyecto de Haris Bin Amir.

BiLSTM es una variante de las redes LSTM² que procesa la información en dos direcciones, desde el inicio y desde el final de la secuencia. Gracias a ello, es capaz de aprovechar tanto el contexto anterior como el posterior de cada muestra, mejorando el reconocimiento de secuencias temporales. Este algoritmo se utiliza en el proyecto de Urav Dalal.

Attention-BiLSTM amplía el modelo BiLSTM mediante la incorporación de un mecanismo de atención que asigna mayor importancia a las partes más relevantes de la secuencia durante el proceso de clasificación. Este enfoque aparece en el trabajo de Ang Ji.

Los métodos basados en umbrales consisten en establecer reglas fijas sobre los valores de los sensores para determinar el signo realizado. Aunque son sencillos de implementar y presentan un coste computacional muy reducido, su capacidad de generalización es limitada y son especialmente sensibles al ruido. Este enfoque fue empleado en los proyectos GuanTELECO y SignGlove.

Las redes neuronales convolucionales (CNN) son modelos ampliamente utilizados para el reconocimiento de patrones. Mediante capas de convolución y agrupación (*pooling*), son capaces de extraer automáticamente características relevantes sin necesidad de diseñarlas manualmente. Este tipo de redes se encuentra en el trabajo de Feng Wen y en el de Swaroop Gudi.

K-Nearest Neighbour (KNN) es un algoritmo basado en distancias que clasifica una muestra según la clase de sus vecinos más cercanos en el espacio de características. Este método fue utilizado en el proyecto de Gill y en el de Wania Khan (de la Universidad Bahria).

El perceptrón multicapa (MLP) es una red neuronal formada por una capa de entrada, una o varias capas ocultas y una capa de salida. Durante el entrenamiento, ajusta los pesos de las conexiones entre neuronas para construir un modelo predictivo capaz de clasificar nuevas muestras.

Finalmente, *Support Vector Machine (SVM)* es un algoritmo que busca un hi-

¹Modelos supervisados que permiten tomar decisiones mediante una estructura jerárquica. Son fáciles de interpretar, pero pueden presentar sobreajuste si no se controlan adecuadamente.

²Memoria a corto y largo plazo; se trata de una red neuronal recurrente que utiliza información pasada para mejorar el rendimiento; es útil para mantener información a largo plazo.

perplano óptimo capaz de separar las distintas clases, maximizando la distancia entre ellas. Este enfoque fue utilizado en el proyecto desarrollado por Wania Khan.

Con el fin de ampliar la comparación, también se incluyeron la regresión logística y Gaussian Naive Bayes, dos algoritmos clásicos ampliamente utilizados como referencia en problemas de clasificación supervisada.

La regresión logística es un modelo lineal de clasificación que estima la probabilidad de pertenencia de una muestra a cada clase mediante una función logística. Destaca por su sencillez, rapidez de entrenamiento y facilidad de interpretación, por lo que se emplea habitualmente como modelo de referencia en problemas de clasificación supervisada.

Gaussian Naive Bayes es un clasificador probabilístico basado en el teorema de Bayes que asume independencia entre las características y una distribución gaussiana de los datos. Gracias a su reducido coste computacional y a su rapidez de entrenamiento, constituye una alternativa eficiente para tareas de clasificación.

De los algoritmos descritos anteriormente, para la comparación experimental realizada en este trabajo se seleccionaron únicamente modelos pertenecientes al ámbito de *machine learning* clásico. En concreto se evaluaron *Random Forest*, árboles de decisión, regresión logística, KNN, SVM y *Gaussian Naive Bayes*. La elección de estos modelos se debe a su reducido coste computacional, su adecuación al tamaño del conjunto de datos disponible y a que representan diferentes estrategias de clasificación, permitiendo realizar una comparación amplia y objetiva de su rendimiento.

La Tabla 7.1 resume los algoritmos evaluados, indicando la familia a la que pertenecen y si requieren un proceso previo de escalado de las características para obtener un rendimiento adecuado.

Tabla 7.1

Algoritmos evaluados durante la comparación de modelos.

Algoritmo	Familia	Requiere escalado
<i>Random Forest</i>	Ensemble	No
<i>Decision Tree</i>	Árbol de decisión	No
Regresión logística	Modelo lineal	Sí
<i>K-Nearest Neighbors (KNN)</i>	Basado en instancias	Sí
<i>Support Vector Machine (SVM)</i>	Vectores soporte	Sí
<i>Gaussian Naive Bayes</i>	Probabilístico	No

7.2.2. Conjunto de datos

Los algoritmos descritos anteriormente fueron evaluados utilizando el conjunto de datos generado específicamente para este trabajo. El conjunto de datos utilizado para la comparación está formado por 4800 instancias (actualmente 5860 instancias), distribuidas en 32 clases, que corresponden a las distintas clases consideradas en este trabajo, incluyendo las letras del alfabeto de la Lengua de Signos Española (LSE) y varios signos adicionales. Cada instancia se obtuvo a partir de una ventana temporal de

diez muestras, capturadas por el guante sensorizado, y está compuesta por una etiqueta de clase y 18 características extraídas de los sensores de flexión y del sensor inercial MPU6050.

La Tabla 7.2 muestra un ejemplo de varias instancias correspondientes a la letra A. Dado que el conjunto completo está formado por 19 columnas, con el objetivo de facilitar la lectura, únicamente se representan algunas de las características utilizadas durante el entrenamiento de los modelos, mientras que la estructura completa del conjunto de datos se describe en el Capítulo 5.

Tabla 7.2

Ejemplo de tres instancias correspondientes a la letra A del conjunto de datos.

Label	Pulgar	Índice	Corazón	Anular	Meñique	Pitch	Roll
A	2811.2	2689.1	2725.3	2921.4	3772.6	-1.387	-2.939
A	2807.7	2682.6	2717.8	2914.0	3768.3	-1.355	-2.761
A	2805.7	2676.7	2715.6	2915.2	3762.6	-1.376	-2.747

7.2.3. Optimización de hiperparámetros

Antes de realizar la comparación entre los diferentes algoritmos, se llevó a cabo un proceso de optimización de hiperparámetros con el objetivo de obtener una configuración adecuada para cada modelo. El rendimiento de un algoritmo de aprendizaje automático puede variar significativamente en función de los hiperparámetros.

Para ello, se empleó la técnica *Randomized Search Cross Validation* [17], proporcionada por la biblioteca *scikit-learn* [18]. Este método consiste en evaluar un conjunto de combinaciones de hiperparámetros seleccionadas de forma aleatoria a partir de un espacio de búsqueda previamente definido. Cada combinación es entrenada y evaluada mediante validación cruzada, seleccionando la mejor combinación de hiperparámetros. El uso de esta técnica presenta la ventaja de reducir considerablemente el coste computacional respecto a una búsqueda exhaustiva de todas las combinaciones posibles, manteniendo al mismo tiempo una elevada posibilidad de encontrar configuraciones de alto rendimiento.

Una vez obtenidos los mejores hiperparámetros para cada algoritmo, estos se utilizaron durante la fase de comparación, asegurando que todos los algoritmos funcionaran en las mejores condiciones.

7.2.4. Validación cruzada

Con el fin de obtener una estimación fiable del rendimiento de cada algoritmo y reducir la influencia de una partición concreta del conjunto de datos, se empleó una estrategia de validación cruzada dividida en diez particiones.

Esta técnica consiste en dividir el conjunto de datos en diez subconjuntos de tamaño similar. En cada iteración, uno de los subconjuntos se utiliza para la evaluación del modelo, mientras que los nueve restantes se destinan al entrenamiento. Este proceso

se repite diez veces, de manera que cada subconjunto actúa una única vez como conjunto de prueba.

Finalmente, las métricas obtenidas en las diez iteraciones se promedian para obtener una estimación representativa del rendimiento de cada algoritmo. Además, se calculó la desviación estándar de dichas métricas con el objetivo de analizar la estabilidad de los modelos frente a las diferentes particiones del conjunto de datos.

La utilización de una validación cruzada estratificada permite aprovechar la totalidad del conjunto de datos durante el proceso de evaluación y proporciona una comparación más robusta y objetiva entre los distintos algoritmos considerados.

7.2.5. Métricas de evaluación

Con el objetivo de comparar el rendimiento de los diferentes algoritmos de clasificación, se emplearon diversas métricas que permiten evaluar tanto la capacidad predictiva de los modelos como su eficiencia computacional [19].

Las métricas consideradas son las siguientes:

- *Accuracy*: Representa la proporción de muestras correctamente clasificadas respecto al total de muestras evaluadas. Constituye una medida global del rendimiento del clasificador.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (7.1)$$

- *Precision*: Mide la proporción de predicciones positivas que son correctas respecto al total de predicciones positivas realizadas. Esta métrica permite evaluar la presencia de falsos positivos en las predicciones del modelo.

$$Precision = \frac{TP}{TP + FP} \quad (7.2)$$

- *Recall*: Indica la capacidad del modelo para identificar las muestras de una determinada clase, evaluando la proporción de verdaderos positivos detectados.

$$Recall = \frac{TP}{TP + FN} \quad (7.3)$$

- *F1-score*: Corresponde a la media armónica entre la precisión (*Precision*) y la sensibilidad (*Recall*), proporcionando una medida equilibrada del rendimiento del clasificador cuando ambas métricas son relevantes.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (7.4)$$

Donde:

- *TP* (*True Positives*): número de verdaderos positivos.

- *TN* (*True Negatives*): número de verdaderos negativos.
- *FP* (*False Positives*): número de falsos positivos.
- *FN* (*False Negatives*): número de falsos negativos.

Además de las métricas de clasificación, también se analizaron dos medidas relacionadas con el costo computacional de cada algoritmo:

- Tiempo de entrenamiento: Tiempo necesario para construir el modelo a partir del conjunto de datos.
- Tiempo de predicción: Tiempo empleado por el modelo previamente entrenado para clasificar nuevas muestras.

Para cada una de estas métricas se calculó tanto su valor medio como su desviación estándar a partir de los resultados obtenidos mediante la validación cruzada. De este modo, fue posible evaluar no solo el rendimiento medio de cada algoritmo, sino también la estabilidad de dicho rendimiento frente a diferentes particiones del conjunto de datos.

7.3. Resultados obtenidos

Una vez finalizado el proceso de optimización de hiperparámetros y aplicada la metodología de evaluación descrita en el apartado anterior, se obtuvieron los resultados correspondientes a cada uno de los algoritmos considerados.

La Tabla 7.3 y la Tabla 7.4 presentan los resultados obtenidos para los seis algoritmos evaluados, incluyendo las métricas de clasificación (*Accuracy*, *Precision*, *Recall* y *F1-score*), así como los tiempos medios de entrenamiento y predicción.

Tabla 7.3

Accuracy y F1-score obtenidos para los algoritmos evaluados.

Modelo	<i>Accuracy</i>	<i>F1-score</i>
<i>Random Forest</i>	0.9974	0.9974
<i>Decision Tree</i>	0.9837	0.9836
Regresión logística	0.9931	0.9931
KNN	0.9658	0.9655
SVM	0.9944	0.9944
<i>Gaussian Naive Bayes</i>	0.9826	0.9826

Tabla 7.4

Tiempos de entrenamiento y predicción de los algoritmos evaluados.

Modelo	<i>Train</i> (s)	<i>Predict</i> (s)
<i>Random Forest</i>	3.099	0.034
<i>Decision Tree</i>	0.060	0.007
Regresión logística	0.190	0.008
KNN	0.003	0.279
SVM	0.080	0.022
<i>Gaussian Naive Bayes</i>	0.005	0.007

Todos los algoritmos evaluados obtienen valores elevados en las métricas de clasificación, alcanzando una exactitud superior al 96 %. No obstante, se aprecian diferencias tanto en la capacidad de clasificación como en el coste computacional asociado al entrenamiento y la predicción. En particular, *Random Forest* presenta los mejores resultados en las métricas de clasificación, mientras que otros algoritmos, como KNN o *Gaussian Naive Bayes*, destacan por sus reducidos tiempos de entrenamiento a costa de una menor precisión. A continuación, se muestran diferentes representaciones gráficas con el fin de facilitar la comparación del rendimiento y la eficacia computacional de cada modelo. Este tipo de comparación entre distintos clasificadores también ha sido empleada en otros sistemas de reconocimiento de lengua de signos [20].

Entre los algoritmos comparados, *Random Forest* obtiene el mayor valor de *Accuracy*, alcanzando una exactitud del 99,74 %. A continuación, se sitúan SVM y la regresión logística, cuyos resultados también son superiores al 99 %. Por otro lado, *Decision Tree*, *Gaussian Naive Bayes* y, especialmente, KNN presentan valores ligeramente inferiores, aunque mantienen un rendimiento satisfactorio.

Las reducidas desviaciones estándar representadas mediante las barras de error evidencian que los resultados obtenidos son consistentes entre las diferentes particiones de la validación cruzada, lo que indica una elevada estabilidad en el proceso de evaluación de los algoritmos.

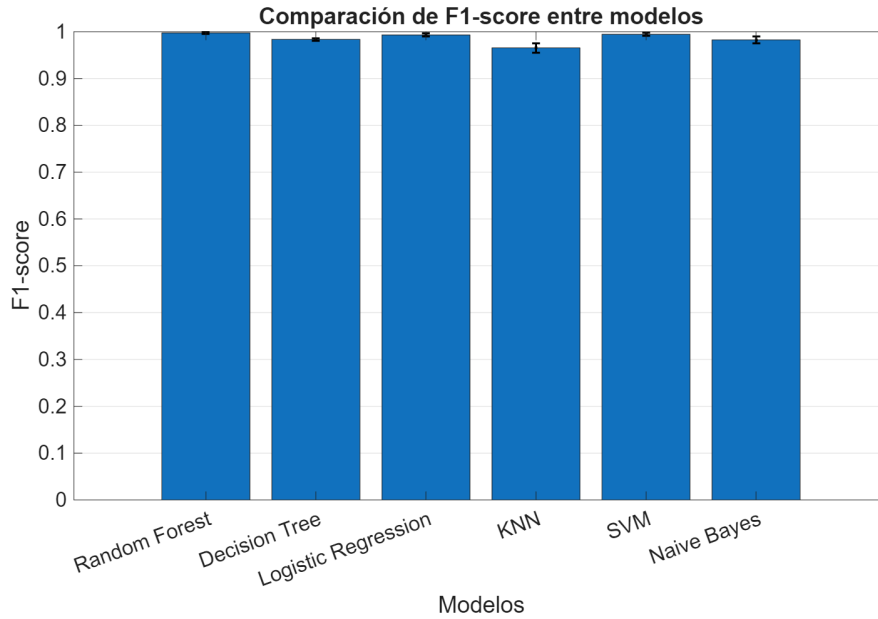


Figura 7.1: $F1$ -score de cada algoritmo.

La Figura 7.1 muestra la comparación del $F1$ -score obtenido por los distintos algoritmos evaluados. Esta métrica proporciona una medida equilibrada del rendimiento del clasificador al combinar la precisión (*Precision*) y la sensibilidad (*Recall*), siendo especialmente útil para valorar el comportamiento global de un modelo.

En este caso, *Random Forest* vuelve a alcanzar un mayor valor de $F1$ -score, seguido de SVM y la regresión logística, cuyos resultados también superan el 99 %. Por su parte, *Decision Tree*, *Gaussian Naive Bayes* y KNN presentan valores inferiores, siendo este último el algoritmo con el menor rendimiento.

Al igual que en la comparación del *Accuracy*, las barras de error presentan una variabilidad reducida entre las distintas particiones de la validación cruzada, lo que pone de manifiesto la estabilidad de los resultados obtenidos para todos los algoritmos evaluados.

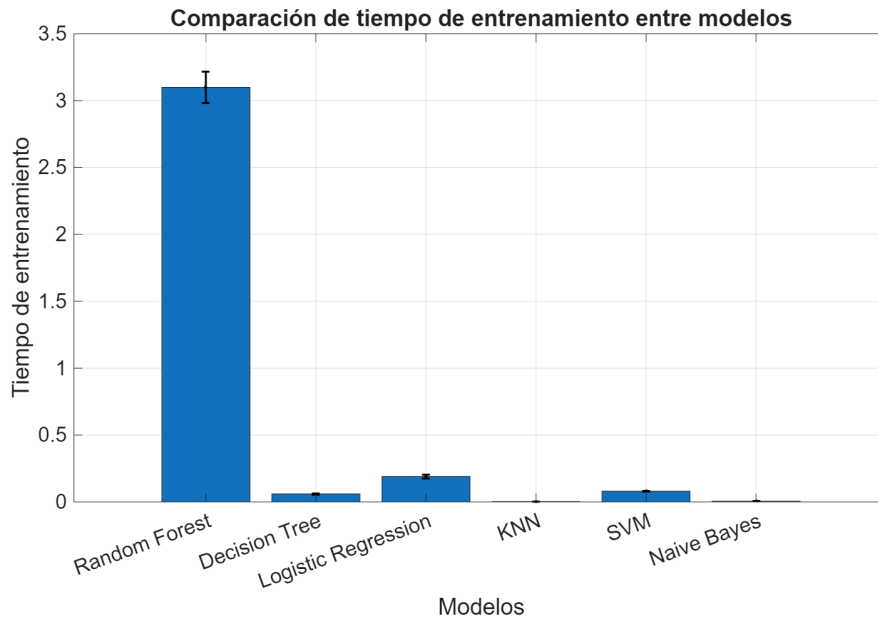


Figura 7.2: Tiempo medio de entrenamiento de cada algoritmo.

La Figura 7.2 presenta el tiempo medio de entrenamiento de los algoritmos evaluados. A diferencia de las métricas de clasificación, en este caso se observan diferencias significativas entre los distintos modelos.

Como puede apreciarse, *Random Forest* requiere el mayor tiempo de entrenamiento, debido a que el modelo debe construir un conjunto de árboles de decisión durante el proceso de aprendizaje. En contraste, algoritmos como KNN y *Gaussian Naive Bayes* presentan tiempos de entrenamiento muy reducidos, ya que su proceso de aprendizaje es considerablemente más sencillo. Por su parte, *Decision Tree*, la regresión logística y SVM muestran tiempos intermedios entre ambos extremos.

Aunque el tiempo de entrenamiento de *Random Forest* es superior al de los demás algoritmos, este proceso únicamente se realiza durante la fase de generación del modelo. Por tanto, este coste computacional no afecta al funcionamiento del sistema durante la traducción de signos.

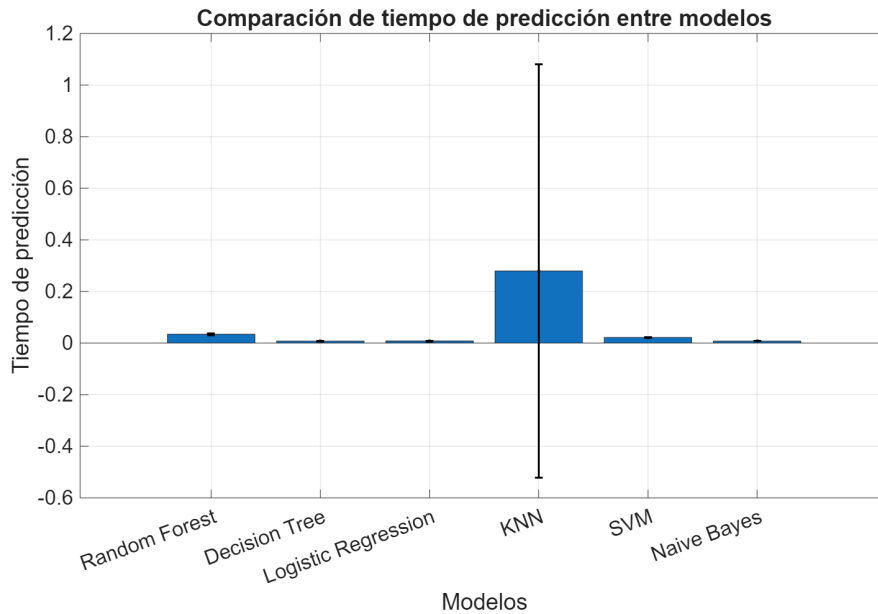


Figura 7.3: Tiempo medio de predicción de cada algoritmo.

La Figura 7.3 muestra el tiempo medio de predicción de los algoritmos evaluados. Esta métrica resulta especialmente relevante en el sistema desarrollado, ya que determina el tiempo necesario para clasificar una nueva muestra una vez que el modelo ya ha sido entrenado.

Los resultados muestran diferencias significativas entre los distintos algoritmos. KNN presenta el mayor tiempo de predicción, debido a que necesita calcular la distancia entre la muestra de entrada y el conjunto de datos de entrenamiento para realizar la clasificación. En el extremo opuesto, *Decision Tree*, *Gaussian Naive Bayes* y la regresión logística obtienen menores tiempos de predicción, mientras que SVM y *Random Forest* presentan tiempos intermedios.

A pesar de que *Random Forest* no es el algoritmo de menor predicción, el valor obtenido resulta reducido y compatible con los requisitos de funcionamiento en tiempo real del sistema desarrollado. Por esta razón, el tiempo de inferencia no supone una limitación para su uso en la aplicación.

Considerando conjuntamente todas las métricas analizadas, *Random Forest* ofrece el mejor equilibrio entre capacidad de clasificación y rendimiento computacional. Por este motivo, este algoritmo fue seleccionado como modelo de clasificación definitivo para el sistema desarrollado, constituyendo la base del proceso de reconocimiento de signos implementado en la aplicación.

8. Aplicación de escritorio

8.1. Descripción general

La aplicación de escritorio constituye el punto de interacción entre el usuario y el sistema desarrollado. Su principal función es facilitar tanto la generación de conjuntos de datos como la traducción de signos en tiempo real mediante una interfaz gráfica sencilla e intuitiva.

La aplicación ha sido desarrollada en Python utilizando la librería *CustomTkinter*, proporcionando una apariencia moderna y una navegación sencilla entre las distintas funcionalidades del sistema. Además, integra la comunicación con el microcontrolador Arduino, el procesamiento de los datos recibidos y la ejecución del modelo de *machine learning* entrenado previamente.

8.2. Arquitectura de la aplicación

La aplicación se organiza de forma modular, separando la interfaz gráfica de la lógica encargada de la adquisición de datos y del control de los procesos internos. Esta organización facilita el mantenimiento del código y permite modificar o ampliar funcionalidades sin afectar al resto del sistema.

app.py

El módulo *app.py* actúa como punto de entrada de la aplicación y gestiona la interfaz gráfica de usuario. Su función principal es implementar la navegación entre las distintas pantallas del sistema, permitiendo la interacción del usuario con las diferentes funcionalidades disponibles.

Para ello, este módulo implementa la lógica de control de la interfaz mediante la librería *CustomTkinter*, encargándose de la creación y gestión de las distintas ventanas que componen la aplicación. Asimismo, gestiona los eventos generados por el usuario, como la selección del modo de funcionamiento o la activación de los distintos procesos, delegando la ejecución de cada funcionalidad en los módulos correspondientes del sistema.

De este modo, *app.py* coordina el flujo general de la aplicación, centralizando la navegación entre las diferentes pantallas y sirviendo como punto de enlace entre la interfaz gráfica y el resto de los módulos encargados de la adquisición, procesamiento y clasificación de los datos.

Lector_UI.py

El módulo *Lector_UI.py* se encarga de la adquisición de datos procedentes del microcontrolador Arduino durante la fase de recolección. Su principal función es gestionar la captura de muestras procedentes de los sensores del guante y coordinar el flujo de adquisición hasta su almacenamiento.

Para ello, los datos recibidos desde el microcontrolador son procesados en tiempo real y agrupados en ventanas temporales de tamaño fijo, con el objetivo de extraer información más robusta y representativa del movimiento. El flujo de adquisición se controla mediante mecanismos de sincronización basados en eventos, que permiten pausar y reanudar la lectura entre cada conjunto de muestras, facilitando la preparación del usuario antes de cada registro.

Una vez obtenidas las ventanas de datos, estas son procesadas mediante las funciones auxiliares del módulo *FuncionesLector.py*, encargado del preprocesamiento de las señales y de la extracción de características. Posteriormente, los vectores de características generados se almacenan en archivos CSV, donde cada fila representa una muestra asociada al signo seleccionado previamente por el usuario.

Además, el módulo mantiene información sobre el estado de la adquisición, como el progreso de las muestras y la última lectura realizada, permitiendo que esta información se muestre en tiempo real por la interfaz gráfica.

8.3. Diseño de la interfaz de usuario

La interfaz de usuario del sistema ha sido diseñada con el objetivo de proporcionar un entorno sencillo, intuitivo y accesible que facilite la interacción del usuario con las distintas funcionalidades de la aplicación. Para su implementación, se ha utilizado la librería *CustomTkinter*, lo que ha permitido desarrollar una interfaz gráfica coherente y adaptada al flujo de trabajo del sistema.

En cuanto al diseño visual, la interfaz mantiene una estética simple y uniforme en todas las ventanas de la aplicación. Para el fondo, se utiliza el color predeterminado de *CustomTkinter*, mientras que los elementos interactivos utilizan un color verde que contrasta con el fondo. Por otro lado, el texto se muestra en color blanco utilizando la tipografía *Arial*. En conjunto, se ha priorizado un diseño limpio y funcional, evitando el uso de elementos gráficos innecesarios.

La aplicación se organiza en varias pantallas independientes, cada una de ellas asociada a una funcionalidad específica del sistema. La pantalla principal o menú principal actúa como punto de entrada a la aplicación, desde la cual el usuario puede acceder a los distintos modos de funcionamiento. A partir de esta, se dispone de una ventana dedicada al proceso de lectura de datos (ver Figura 8.1), en la cual el usuario deberá introducir qué palabra o letra se va a almacenar antes de iniciar el proceso, funcionando como un menú previo al proceso de recolección de datos.

A continuación, se dispone de una ventana de lectura (ver Figura 8.2), la cual permite gestionar el flujo del proceso de adquisición de datos. Esta pantalla incluye

dos botones principales: *Salir*, que permite regresar al menú de lectura, y *Continuar*, que inicia una nueva captura de datos para cada signo. De esta manera, el usuario tendrá tiempo para prepararse, al tiempo que se limpia el *buffer*, evitando que los datos recogidos correspondan a muestras previas o a información residual de capturas anteriores. Al finalizar la lectura, se accede a una pantalla que notifica al usuario que el proceso ha finalizado. Desde esta pantalla, el usuario únicamente puede regresar al menú principal.

Por otro lado, la pantalla del menú de traducción (ver Figura 8.3), accesible desde el menú principal, permite al usuario establecer la conexión con el microcontrolador y gestionar el proceso de reconocimiento de signos. Esta ventana dispone de un campo de texto en el que se introduce la dirección *IP* en caso de utilizar conexión WiFi, así como de dos botones para la selección del modo de conexión: conexión WiFi y conexión local, que realizan la conexión mediante el puerto serie USB. Además, se incluye un botón de inicio de traducción, que únicamente puede activarse una vez establecida correctamente una conexión previa, y un botón de *Volver* para regresar al menú principal.

A continuación, la pantalla de traducción (ver Figura 8.4) permite visualizar los resultados obtenidos durante el proceso de reconocimiento de signos. En esta ventana se muestra en tiempo real la predicción principal del modelo, correspondiente al signo con mayor probabilidad según el modelo *Random Forest*, así como los tres candidatos más probables asociados a la clasificación realizada. Además, la interfaz dispone de un único botón de salida que permite finalizar y volver a la pantalla principal.

Por último, se dispone de una pantalla de error (ver Figura 8.5) que informa al usuario cuando se produce un problema con los datos recibidos de los sensores. Esta pantalla muestra un mensaje que recomienda reiniciar el Arduino y comprobar las conexiones de los cables. El acceso a esta pantalla se realiza cuando el sensor MPU6050 deja de proporcionar valores válidos (únicamente devuelve 0) o cuando alguno de los sensores de flexión alcanza valores anómalos (4095).

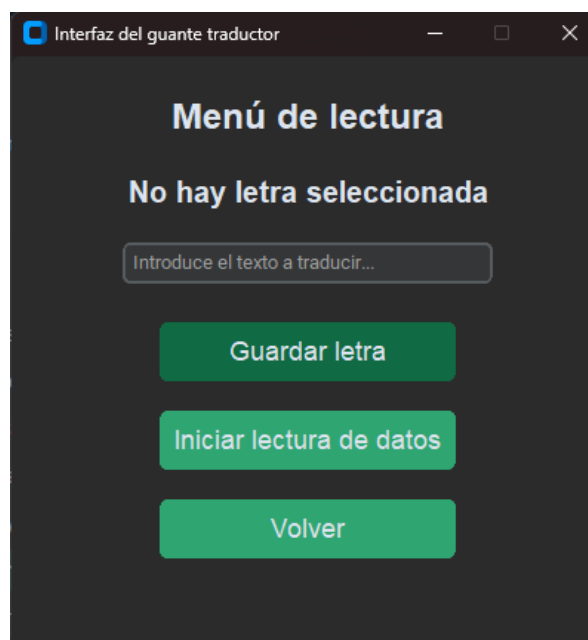


Figura 8.1: Pantalla del menú de lectura.

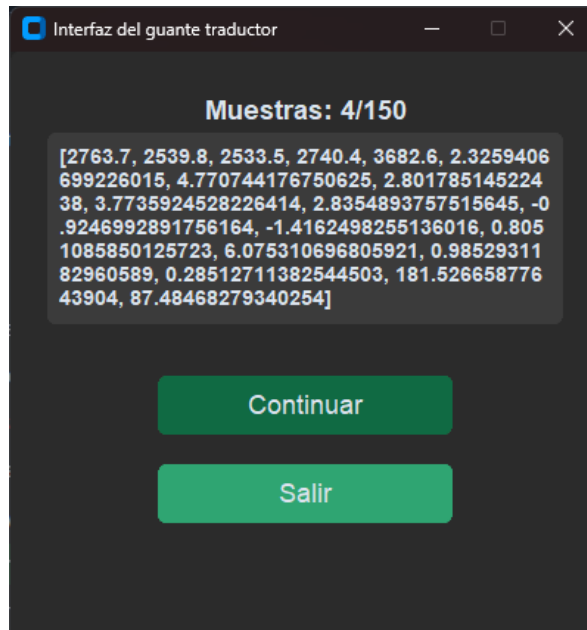


Figura 8.2: Pantalla de recolección de datos.

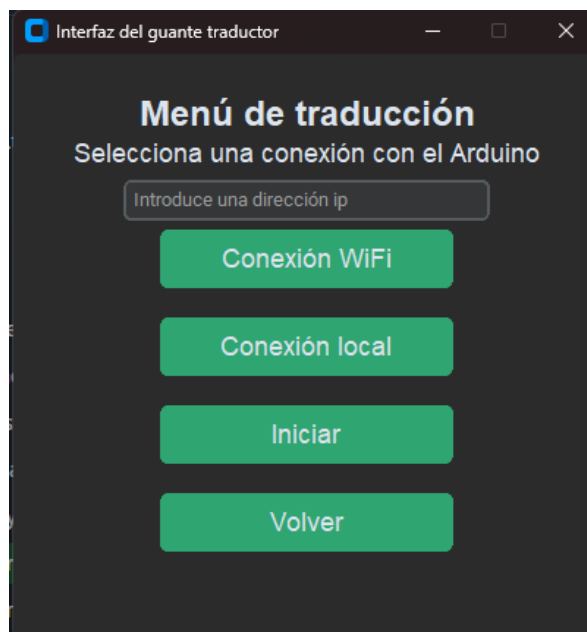


Figura 8.3: Pantalla del menú de traducción.

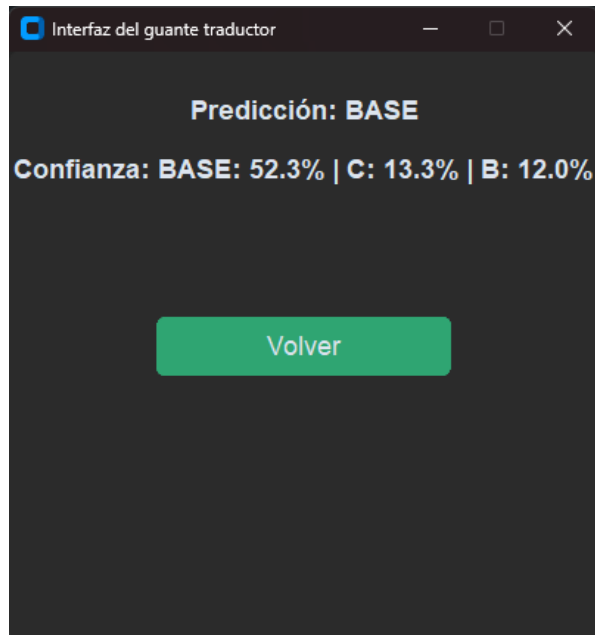


Figura 8.4: Pantalla de traducción en tiempo real.

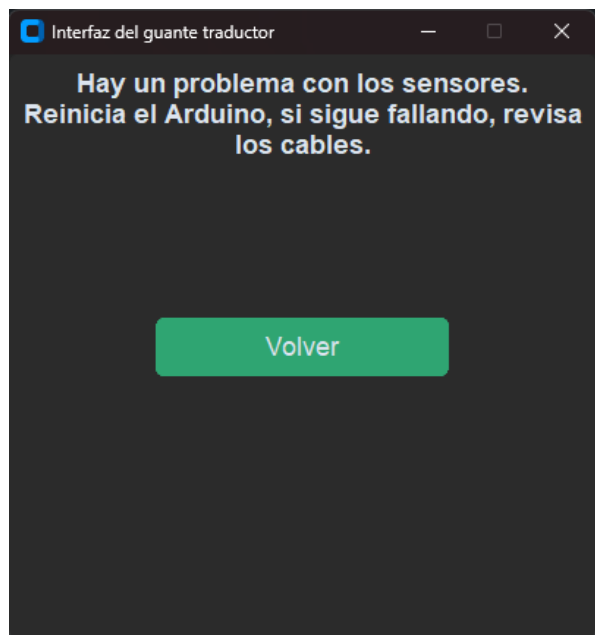


Figura 8.5: Pantalla de error.

9. Pruebas del sistema

En este capítulo se presenta el plan de pruebas del sistema de información, incluyendo los diferentes tipos de pruebas que se han llevado a cabo, ya sea de forma manual (mediante listas de comprobación) o automatizadas, mediante algún software específico de pruebas.

9.1. Estrategia

Las pruebas realizadas tienen como objetivo verificar el correcto funcionamiento del sistema en todos sus niveles, desde el comportamiento individual de los distintos módulos hasta la validación del sistema completo. Para ello, se han llevado a cabo pruebas unitarias y de integración, registrándose únicamente aquellas consideradas relevantes para demostrar el cumplimiento de los requisitos definidos durante la fase de análisis.

Durante el desarrollo del proyecto, cada vez que se realizó una modificación sobre alguno de los módulos del sistema, se repitieron las pruebas afectadas con el objetivo de comprobar que los introducidos no alteraban las funcionalidades previamente validadas. De este modo, se garantizó que la corrección de incidencias no provocara nuevos errores en el resto de la aplicación.

Todas las pruebas fueron diseñadas, ejecutadas y evaluadas por el autor del proyecto, utilizando el mismo entorno de hardware y software empleado durante el desarrollo. Dicho entorno está compuesto por un ordenador con sistema operativo Windows 11, Python 3.14 como entorno de ejecución y el guante sensorizado basado en Arduino, conectado mediante puerto USB o conexión WiFi, según la funcionalidad evaluada.

9.2. Pruebas unitarias

Las pruebas unitarias se han realizado con el objetivo de verificar el correcto funcionamiento de los distintos módulos de software de forma independiente, permitiendo detectar posibles errores antes de su integración con el resto del sistema. Para su implementación, se ha utilizado la herramienta *pytest*, ampliamente empleada en el desarrollo de aplicaciones Python debido a su sencillez y flexibilidad.

Se han diseñado pruebas unitarias para aquellas funciones que implementan lógica propia, evitando aquellas que dependen de la interfaz gráfica para su funcionamiento o que se ejecutan en un bucle infinito. De este modo, las pruebas se centraron en comprobar el comportamiento de las funciones encargadas del procesamiento de datos, la preparación del conjunto de entrenamiento, la gestión del modelo de *machine learning* y las funciones auxiliares de los distintos módulos del sistema.

Se han desarrollado pruebas para todos los módulos que afectan directamente al flujo de ejecución del sistema, *app.py*, *FuncionesArduino.py*, *FuncionesLector.py*, *FuncionesRF.py* y *Lector_UI.py*, verificando tanto el funcionamiento esperado como el tratamiento de diferentes casos límite, utilizando objetos simulados para aislar las dependencias externas.

Tras la ejecución del conjunto de pruebas mediante *pytest* (ver Figura 9.1), todas ellas fueron superadas satisfactoriamente, confirmando el correcto funcionamiento de las funciones evaluadas antes de proceder con las pruebas de integración del sistema.

```
PS C:\Users\David\Desktop\TFG\TFG\TFG> python -m pytest
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\David\Desktop\TFG\TFG\TFG
collected 29 items

tests\test_FuncionesArduino.py .... [ 13%]
tests\test_FuncionesLector.py ..... [ 37%]
tests\test_FuncionesRF.py ..... [ 56%]
tests\test_LectorUI.py ..... [ 89%]
tests\test_app.py ... [100%]

===== 29 passed in 2.30s =====
```

Figura 9.1: Resultado de ejecutar Pytest.

Los tests implementados se encuentran dentro del proyecto, en la carpeta denominada *tests*.

9.3. Pruebas de integración

Las pruebas de integración se realizaron con el objetivo de verificar el correcto funcionamiento de la interacción entre los distintos módulos que componen el sistema. A diferencia de las pruebas unitarias, en este nivel se evaluó el comportamiento conjunto de varios componentes.

Para ello, se ejecutaron diferentes escenarios que abarcan desde la adquisición del guante sensorizado hasta la visualización de la predicción obtenida por el modelo de *machine learning*, verificando en cada caso el comportamiento esperado del sistema.

Conexión entre la interfaz gráfica y el Arduino

Objetivo: Verificar que la aplicación establece correctamente la comunicación con el microcontrolador Arduino mediante el puerto serie, permitiendo la transmisión de los datos de los sensores.

Procedimiento

1. Iniciar la aplicación.
2. Acceder al menú de traducción.
3. Conectar el Arduino al equipo mediante el puerto USB
4. Seleccionar la opción de conexión local.

5. Comprobar que la aplicación establece correctamente la conexión con el dispositivo. La interfaz mostrará dicha información.

Resultado esperado: La conexión debe establecerse correctamente, mostrando en la interfaz un mensaje de confirmación y permitiendo iniciar el proceso de traducción de signos.

Resultado obtenido: La conexión con el Arduino mediante el puerto serie funciona correctamente; se muestra en la interfaz y permite la comunicación con el guante y el inicio del proceso de traducción.

Procesamiento de los datos recibidos

Objetivo: Verificar que los datos enviados por el Arduino son recibidos correctamente por la aplicación, procesados y transformados en un vector de características válido para su posterior validación.

Procedimiento.

1. Establecer la conexión con el Arduino.
2. Iniciar el proceso de traducción.
3. Realizar diferentes signos con el guante sensorizado.
4. Comprobar que las muestras recibidas son procesadas correctamente por la aplicación sin producir errores.

Resultado esperado: Los datos recibidos deben convertirse correctamente a valores numéricos, agruparse en ventanas de muestras y generar un vector de características válido para el modelo de *machine learning*.

Resultado obtenido: Los datos fueron procesados correctamente durante toda la ejecución de la prueba, generándose los vectores de características esperados. Sin embargo, hay ocasiones en las que se produce un error al intentar transformar un tipo de dato *float* a *int*, aunque este problema está manejado mediante el uso de excepciones de Python.

Generación del conjunto de datos

Objetivo: Verificar que las características extraídas de las muestras del guante se almacenan correctamente en el conjunto de datos utilizado para el entrenamiento del modelo.

Procedimiento.

1. Acceder al modo de lectura de la aplicación.
2. Seleccionar la etiqueta correspondiente al signo que se desea registrar.
3. Realizar el signo hasta completar el número de muestras requerido por la aplicación.

4. Comprobar el contenido del archivo CSV generado.

Resultado esperado: El sistema debe generar o actualizar el archivo CSV correspondiente, almacenando correctamente cada muestra junto con su etiqueta y las características calculadas.

Resultado obtenido: El conjunto de datos se genera correctamente, registrándose todas las muestras esperadas con su correspondiente etiqueta y sin detectarse errores en la escritura del archivo CSV.

Entrenamiento del modelo de *machine learning*

Objetivo: Verificar que el conjunto de datos generado puede utilizarse correctamente para entrenar el modelo de *machine learning* sin producir errores durante el proceso.

Procedimiento.

1. Seleccionar el conjunto de datos previamente generado.
2. Iniciar el proceso de entrenamiento desde la aplicación.
3. Esperar a que finalice el entrenamiento del modelo.
4. Comprobar que el modelo se genera correctamente y está preparado para realizar predicciones.

Resultado esperado: El conjunto de datos debe cargarse correctamente, realizar el preprocesamiento de los datos y completar el entrenamiento del modelo sin incidentes.

Resultado obtenido: El modelo fue entrenado correctamente a partir del conjunto de datos, quedando disponible para realizar las predicciones en tiempo real.

Traducción de signos en tiempo real

Objetivo: Verificar el funcionamiento conjunto de todos los módulos implicados en la traducción de signos, desde la adquisición de datos hasta la visualización de la predicción en la interfaz gráfica.

Procedimiento.

1. Establecer la conexión con el Arduino.
2. Seleccionar el conjunto de datos para entrenar el modelo.
3. Iniciar el proceso de traducción.
4. Realizar distintos signos con el guante sensorizado.
5. Comprobar que la predicción mostrada en la interfaz corresponde al signo realizado.

Resultado esperado: El sistema debe recibir correctamente los datos de los sensores, preprocesarlos y utilizarlos en el modelo de *machine learning* entrenado para realizar las predicciones en tiempo real. Se espera que los resultados no sean perfectos ni inestables.

Resultado obtenido: La traducción de signos se realizó correctamente durante toda la prueba, mostrando en la interfaz las predicciones generadas por el modelo en tiempo real. Sin embargo, al tocar el giroscopio, este deja de proporcionar datos al sistema.

9.4. Pruebas de sistema

En esta sección se presentan las pruebas realizadas sobre el sistema completo para verificar el cumplimiento de los requisitos definidos durante la fase de análisis. Estas pruebas se dividen en pruebas funcionales y pruebas no funcionales.

9.4.1. Pruebas funcionales

En esta sección se describen las pruebas realizadas sobre el sistema con el objetivo de verificar que las principales funcionalidades del sistema cumplen con los requisitos funcionales definidos durante la fase de análisis. Estas pruebas se llevaron a cabo desde el punto de vista del usuario, comprobando el correcto funcionamiento de los distintos flujos de uso de la aplicación y validando que los resultados obtenidos se corresponden con el comportamiento esperado.

Creación de un subconjunto de datos

Objetivo: Verificar que el sistema permite al usuario generar correctamente un conjunto de datos para el entrenamiento del modelo de *machine learning* a partir de los signos realizados con el guante.

Procedimiento.

1. Iniciar la aplicación.
2. Acceder al menú de recolección de datos.
3. Introducir la etiqueta correspondiente al signo que se desea registrar.
4. Iniciar el proceso de captura de datos.
5. Realizar el signo solicitado hasta completar el número de muestras requerido.
6. Comprobar el archivo CSV generado.

Resultado esperado: El sistema debe capturar los datos enviados por el guante, procesarlos y almacenarlos correctamente en un archivo CSV junto con la etiqueta seleccionada por el usuario.

Resultado obtenido: El sistema completó correctamente la adquisición de datos, generando el archivo CSV con las muestras esperadas y con la etiqueta seleccionada por el usuario como nombre.

Traducción de signos en tiempo real

Objetivo: Verificar que el sistema es capaz de reconocer un signo realizado por el usuario y mostrar la traducción correspondiente en tiempo real.

Procedimiento.

1. Iniciar la aplicación.
2. Acceder al menú de traducción.
3. Establecer la conexión con el Arduino.
4. Seleccionar el conjunto de datos para entrenar el modelo.
5. Iniciar el proceso de traducción.
6. Realizar distintos signos con el guante sensorizado.
7. Comprobar que la predicción mostrada en la interfaz corresponde al signo realizado.

Resultado esperado: El sistema debe recibir y procesar las muestras del guante, clasificarlas mediante el modelo de *machine learning* y mostrar en la interfaz la representación textual del signo reconocido.

Resultado obtenido: El sistema recibe los datos, los procesa y realiza las predicciones en tiempo real, mostrando la información en la interfaz gráfica. Sin embargo, los signos tardan en detectarse debido a la velocidad a la que se reciben los datos del microcontrolador. Además, se ha observado que las letras $\ddot{F}$ y $\ddot{T}$ son indistinguibles para el sistema, debido a que ambas presentan un patrón de flexión de los dedos muy similar.

Selección del método de conexión

Objetivo: Verificar que el sistema permite establecer correctamente la comunicación con el guante sensorizado utilizando los métodos de conexión disponibles.

Procedimiento.

1. Iniciar la aplicación.
2. Acceder al menú de traducción.
3. Seleccionar la conexión mediante puerto serie y comprobar que la conexión se establece correctamente.
4. Repetir el proceso utilizando la conexión mediante WiFi.
5. Verificar que, en ambos casos, se muestre si la conexión ha sido exitosa y si se puede iniciar el proceso de traducción.

Resultado esperado: El sistema debe establecer la comunicación correctamente con el Arduino utilizando el método de conexión seleccionado por el usuario, permitiendo acceder posteriormente al proceso de traducción.

Resultado obtenido: Tanto la conexión mediante puerto serie como la conexión mediante WiFi se establecieron correctamente (aunque para ello hay que configurar las instrucciones del microcontrolador), permitiendo la recepción de datos de los sensores y el inicio del proceso de traducción sin incidencias.

Gestión de errores durante la utilización del sistema

Objetivo: Verificar que el sistema detecte y gestione correctamente las situaciones de error que pueden producirse durante su funcionamiento, informando al usuario y evitando un comportamiento inesperado.

Procedimiento.

1. Iniciar la aplicación.
2. Establecer conexión con el microcontrolador.
3. Iniciar el proceso de lectura o de traducción.
4. Desconectar un sensor de flexión o interactuar con el giroscopio para provocar un error.

Resultado esperado: Al provocar un error, la aplicación mostrará una pantalla en la que se explicará al usuario que se ha producido un error, deteniendo cualquier proceso que estaba en ejecución.

Resultado obtenido: Al obtener datos erróneos de los sensores, el sistema los detecta, detiene cualquier proceso y muestra correctamente la pantalla de error. Además, también gestiona el problema de desconectar el microcontrolador durante el proceso de traducción o lectura.

9.4.2. Pruebas no funcionales

Estas pruebas tienen como objetivo verificar que el sistema cumple con los requisitos no funcionales definidos durante la fase de análisis. Para ello, se evaluaron aspectos relacionados con el rendimiento, la compatibilidad, la fiabilidad y la usabilidad de la aplicación, comprobando que el sistema mantiene un comportamiento estable y adecuado durante su funcionamiento.

Rendimiento en tiempo real

Objetivo: Evaluar el rendimiento del sistema durante la traducción de signos en tiempo real, verificando que el procesamiento de datos y la generación de las predicciones se realicen de forma fluida y sin retardos perceptibles para el usuario.

Procedimiento.

1. Iniciar la aplicación y establecer la conexión con el Arduino.
2. Seleccionar un conjunto de datos previamente generado para entrenar el modelo de *machine learning*.
3. Realizar distintos signos de forma continua durante varios minutos, observando el comportamiento de la aplicación.

Resultado esperado: El sistema debe mostrar las predicciones en la interfaz gráfica de manera fluida, sin bloqueos ni retardos apreciables.

Resultado obtenido: El sistema funciona correctamente los primeros segundos; al pasar unos segundos, disminuye mínimamente la velocidad de predicción y obtención de datos, por lo que hay un pequeño espacio de tiempo entre el signo y la traducción. Este retraso es perceptible para el usuario, aunque no afecta demasiado al funcionamiento.

Compatibilidad con los métodos de conexión

Objetivo: Verificar que el sistema es compatible con los métodos de conexión implementados para la comunicación con el Arduino, permitiendo su funcionamiento tanto mediante el puerto serie como a través de la conexión WiFi.

Procedimiento.

1. Iniciar la aplicación.
2. Establecer la conexión con el Arduino mediante el puerto serie y comprobar el funcionamiento del proceso de traducción.
3. Establecer la conexión con el Arduino mediante WiFi y comprobar el funcionamiento del proceso de traducción.

Resultado esperado: La traducción en tiempo real funciona correctamente independientemente de la conexión seleccionada.

Resultado obtenido: El sistema funcionó correctamente tanto mediante conexión por puerto serie como mediante conexión WiFi. No se nota diferencia entre ambos métodos.

Fiabilidad ante errores de comunicación

Objetivo: Evaluar la capacidad del sistema para detectar y gestionar errores producidos durante la comunicación con el Arduino, evitando bloqueos o comportamientos inesperados.

Procedimiento.

1. Iniciar la aplicación y establecer una conexión con el Arduino.

2. Simular diferentes situaciones de error, como introducir una dirección IP incorrecta, desconectar el Arduino durante la ejecución o provocar la recepción de datos inválidos.
3. Observar la respuesta de la aplicación ante cada una de las situaciones anteriores.

Resultado esperado: El sistema debe detectar las situaciones de error, informar al usuario y evitar que la aplicación continúe funcionando en un estado inconsistente, evitando cualquier bloqueo.

Resultado obtenido: Durante las pruebas, el sistema detectó los errores de introducir una dirección IP incorrecta, provocar datos erróneos de los sensores y la desconexión del Arduino.

Usabilidad de la interfaz gráfica

Objetivo: Evaluar la facilidad de uso de la aplicación, verificando que el usuario pueda acceder y utilizar las distintas funcionalidades mediante la interfaz gráfica de forma sencilla e intuitiva.

Procedimiento.

1. Iniciar la aplicación.
2. Navegar por los diferentes menús de la interfaz.
3. Realizar el proceso completo de creación de un conjunto de datos y de traducción de signos utilizando únicamente los controles disponibles en la aplicación.
4. Comprobar que el sistema proporciona información sobre el estado de las operaciones y guía al usuario durante el proceso.

Resultado esperado: La interfaz debe permitir al usuario acceder a todas las funcionalidades del sistema de manera intuitiva, proporcionando mensajes informativos y evitando acciones que puedan provocar un uso incorrecto de la aplicación. A excepción del método de conexión con el Arduino, ya que este depende además de las instrucciones asignadas al microcontrolador.

Resultado obtenido: La interfaz permitió realizar todas las operaciones previstas sin dificultades, proporcionando información sobre el estado de la aplicación y facilitando la interacción con la interfaz gráfica en todo momento.

Parte III

Epílogo

10. Conclusiones

En este último capítulo, se detallan las lecciones aprendidas tras el desarrollo del presente proyecto y se identifican las posibles oportunidades de mejora sobre el software desarrollado.

10.1. Objetivos alcanzados

El objetivo de este proyecto era desarrollar un prototipo funcional de un guante sensorizado capaz de reconocer signos de la lengua de signos española mediante técnicas de *machine learning*, permitiendo traducir dichos signos a texto de forma automática.

Para alcanzar este objetivo, se han completado satisfactoriamente las diferentes etapas del proyecto. En primer lugar, se ha diseñado y construido un guante integrando cinco sensores de flexión y una unidad de medida inercial, lo que permite la captura de información sobre la posición de los dedos y la orientación de la mano. Además, el diseño de una placa PCB permitió integrar y organizar los componentes electrónicos en un único soporte, reduciendo el cableado y mejorando la comodidad y la libertad de movimiento del usuario.

Por otro lado, se ha desarrollado una aplicación de escritorio encargada de gestionar la comunicación con el dispositivo, facilitando la adquisición de datos, su almacenamiento y la interacción con el usuario mediante una interfaz gráfica.

Otro de los objetivos alcanzados ha sido la creación de un conjunto de datos propio, debido a la inexistencia de bases de datos públicas adaptadas a las necesidades del proyecto. Para ello, se implementó un sistema de captura que permitió registrar múltiples muestras de cada signo, obteniendo un conjunto de datos adecuado para el entrenamiento de modelos de *machine learning*.

Asimismo, se ha realizado un estudio comparativo entre distintos algoritmos de *machine learning*, evaluando su rendimiento mediante validación cruzada y diferentes métricas de clasificación. A partir de los resultados, se seleccionó *Random Forest* como el modelo más adecuado, al ofrecer el mejor equilibrio entre precisión, robustez y tiempo de predicción.

Finalmente, una vez seleccionado el modelo de predicción, se consiguió que la aplicación realizara predicciones en tiempo real a partir de los datos proporcionados por el guante. Las pruebas realizadas demuestran que el sistema es capaz de reconocer correctamente los signos contemplados en el conjunto de datos con una elevada precisión, validando la viabilidad del enfoque propuesto.

En conjunto, se puede concluir que se han alcanzado los objetivos planteados al inicio del trabajo, obteniendo un prototipo funcional que integra hardware, adquisición de datos, procesamiento de señales, aprendizaje automático e inferencia en tiempo real, constituyendo una base sólida para futuras ampliaciones y mejoras.

10.2. Lecciones aprendidas

La realización de este proyecto ha permitido adquirir conocimientos y competencias tanto técnicas como metodológicas, abarcando distintas áreas de la ingeniería informática y la electrónica.

Desde el punto de vista del hardware, se han adquirido conocimientos relacionados con la selección e integración de sensores, el montaje de circuitos electrónicos, el diseño y manipulación de una placa PCB, y el uso de soldadores, lo que permite obtener un dispositivo compacto, fiable y cómodo para el usuario. Asimismo, se ha profundizado en el uso de microcontroladores y en la comunicación entre dispositivos mediante conexión USB y redes WiFi.

En el ámbito del desarrollo de software, el proyecto ha supuesto una oportunidad para diseñar una aplicación completa en Python, implementando una interfaz gráfica, sistemas de adquisición de datos, procesamiento de señales y comunicación con dispositivos externos. Del mismo modo, se ha adquirido experiencia en el desarrollo de aplicaciones multihilo para garantizar una interacción fluida con el usuario.

Por otra parte, el proyecto ha permitido profundizar en el proceso completo de desarrollo de una solución basada en *machine learning*, desde la generación de un conjunto de datos propio hasta la extracción de características, el entrenamiento y la evaluación de diferentes modelos, y su posterior integración en un sistema de predicción en tiempo real. La ausencia de conjuntos de datos públicos adecuados puso de manifiesto la importancia de diseñar procedimientos de captura de datos robustos y de garantizar la calidad de las muestras obtenidas.

A nivel metodológico, el desarrollo del proyecto ha puesto de manifiesto la importancia de la planificación, la realización de pruebas continuas y la validación de cada uno de los módulos antes de su integración con el resto del sistema. Durante el desarrollo surgieron diversos problemas tanto de hardware como de software que requirieron analizar diferentes alternativas y realizar modificaciones sobre el diseño inicial, permitiendo comprender la naturaleza iterativa del proceso de ingeniería.

En términos de dedicación, el proyecto ha requerido varios meses de trabajo continuado. La mayor parte del tiempo y esfuerzo se ha concentrado en el diseño y construcción del guante sensorizado debido a las dificultades asociadas a la integración de los componentes electrónicos, el diseño de la placa PCB y la resolución de los problemas surgidos durante las distintas fases de prueba. Estas tareas requirieron numerosas iteraciones hasta obtener un prototipo funcional y fiable que sirviera como base para el resto del proyecto. Una vez superada esta etapa, la implementación de la aplicación y la integración del modelo de *machine learning* pudieron abordarse de forma más estructurada.

10.3. Trabajo futuro

Aunque el prototipo desarrollado cumple con los objetivos planteados, existen diversas líneas de mejora que permitirían ampliar sus funcionalidades y aumentar su utilidad práctica. Una de las principales mejoras consiste en incorporar un sistema de salida de audio que convierta automáticamente el texto reconocido en voz, facilitando la comunicación con personas no signantes.

Asimismo, una evolución natural del proyecto sería trasladar el procesamiento y la traducción de los signos al propio microcontrolador, permitiendo que el sistema funcionara de forma autónoma y proporcionando al usuario una mayor libertad de movimiento al eliminar la dependencia de un ordenador. En este escenario, la incorporación de conectividad Bluetooth permitiría transmitir el resultado de la traducción a una aplicación móvil, donde el usuario podría visualizar el texto generado y acceder a funcionalidades adicionales. Por ello, otra línea de trabajo consistiría en migrar la aplicación de escritorio a una aplicación para Android, facilitando el uso del sistema en cualquier entorno y aumentando considerablemente su portabilidad.

Otra línea de investigación consistiría en ampliar las capacidades del sistema para que no se limitara al reconocimiento de signos individuales, sino que fuera capaz de interpretar secuencias de signos y construir frases completas. Para ello, sería necesario desarrollar mecanismos que permitieran identificar el inicio y el final de cada signo, gestionar la separación entre palabras y adaptar la traducción a la estructura propia de la lengua de signos española.

Por último, el sistema podría ampliarse incorporando un mayor número de signos y expresiones, aumentando el vocabulario disponible y permitiendo su utilización en un mayor número de situaciones cotidianas. Del mismo modo, la recopilación de datos procedentes de un mayor número de usuarios contribuiría a mejorar la capacidad de generalización del modelo y su robustez frente a las diferencias entre los distintos signantes.

En conjunto, estas mejoras permitirían evolucionar el prototipo desarrollado hacia una situación más completa, portátil y cercana a un entorno de uso real.

Bibliografía

- [1] Yutong Gu, Hiromasa Oku y Masahiro Todoh. American sign language recognition and translation using perception neuron wearable inertial motion capture system. *Sensors*, 24(2):453, 2024.
- [2] David McNaughton y Janice Light. Communicative competence for individuals who require augmentative and alternative communication: A new definition for a new era of communication? *Augmentative and Alternative Communication*, 30(1): 1–18, 2014.
- [3] Muhammad Uzair Shahid, Muhammad Mahi Mahessar y Haris Bin Amir. Iot-enabled assistive glove for real-time sign language translation using machine learning. *International Journal of Innovations in Science & Technology*, 7(3):1568–1583, 2025.
- [4] Urav Dalal. Deep learning-enabled smart glove for real-time sign language translation. *Journal of Electrical Systems*, 20(10), 2024.
- [5] Ang Ji, Wang Yongzhen, Miao Xin y Fan Tianqi. Dataglove for sign language recognition of people with hearing and speech impairment via wearable inertial sensors. *Sensors*, 23(15), 2023. ISSN 1424-8220.
- [6] Swaroop Gudi, Chinmay Inamdar, Yash Divate y Sunil Tayde. Sign language detection using gloves. *Ijrasnet Journal For Research in Applied Science and Engineering Technology*, 12, 2024.
- [7] Feng Wen, Zixuan Zhang, Tianyiyi He y Chengkuo Lee. Ai enabled sign language recognition and vr space bidirectional communication using triboelectric smart glove. *Nature Communications*, 12(1), 2021.
- [8] M. M. Nwachukwu, C. E. Eze, O. D. Nnorom, H. A. Nwebonyi y C. O. Ezeagwu. Implementing sign language recognition system using flex sensors. *International Journal of Research and Innovation in Applied Science*, pages 73–79, 2024.
- [9] Mohamed Aktham Ahmed, Bilal Bahaa Zaidan y Aws Alaa Zaidan. A review on systems-based sensory gloves for sign language recognition state of the art between 2007 and 2017. *Sensors*, 18(7):2208, 2018.
- [10] Kyung-Won Kim, Mi-So Lee, Bo-Ram Soon, Mun-Ho Ryu y Je-Nam Kim. Recognition of sign language with an inertial sensor-based data glove. *Technology and Health Care*, 24(1), 2016.
- [11] Ruiliang Su, Xiang Cheng, Shuai Cao y Xu Zhang. Random forest-based recognition of isolated sign language subwords using data from accelerometers and surface electromyographic sensors. *Sensors*, 16(1):100, 2016.

- [12] Endang Supriyati y Mohammad Iqbal. Recognition system of indonesia sign language based on sensor and artificial neural network. *Makara Journal of Technology*, 17(1), 2013.
- [13] T. L. Alumona, V. N. Okorogu y E. F. Nworabude. Sign language recognition system using flex sensor network. *International Journal of Research and Innovation in Applied Science*, 8(9):182–187, 2023.
- [14] Anton Yudhana, J. Rahmawan y C. U. P. Negara. Flex sensors and mpu6050 sensors responses on smart glove for sign language translation. *IOP Conference Series: Materials Science and Engineering*, 403:012032, 2018.
- [15] Chenghong Lu, Lei Jing y Shingo Amino. Data glove with bending sensor and inertial sensor based on weighted dtw fusion for sign language recognition. *Electronics*, 12(3):613, 2023.
- [16] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [17] James Bergstra y Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [18] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [19] Oona Rainio, Jarmo Teuho y Riku Klén. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14:6086, 2024.
- [20] İlhan Umut y Ümit Can Kundereli. Novel wearable system to recognize sign language in real time. *Sensors*, 24(14):4613, 2024.

Parte IV

Anexos

A. Manual del desarrollador

A.1. Introducción

El presente manual tiene como objetivo proporcionar la información necesaria para facilitar el mantenimiento, la modificación y la evolución del software desarrollado durante este proyecto. Está dirigido a desarrolladores que deseen comprender la estructura, incorporar nuevas funcionalidades o adaptar el sistema a nuevos requisitos.

El software desarrollado forma parte de un sistema de traducción de lengua de signos española basado en un guante sensorizado. Su función principal es gestionar la comunicación con el dispositivo de adquisición, almacenar los datos capturados, entrenar modelos de *machine learning* y realizar la traducción de los signos en tiempo real mediante un modelo previamente entrenado.

El proyecto se ha desarrollado utilizando Python como lenguaje principal para la aplicación de escritorio, el procesamiento de datos y el uso del modelo de *machine learning*, y Arduino para el firmware del microcontrolador encargado de la adquisición de datos. La arquitectura del sistema se encuentra dividida en módulos independientes, lo que facilita el mantenimiento del código y la incorporación de nuevas funcionalidades.

En los siguientes apartados se describen los requisitos necesarios para preparar el entorno de desarrollo, la organización del proyecto, las consideraciones que deben tenerse en cuenta durante la modificación de código y el procedimiento necesario para construir y mantener la aplicación.

A.2. Preparación del entorno de trabajo

Para poder modificar o ampliar el software desarrollado, es necesario preparar previamente el entorno de trabajo, tanto para la aplicación de escritorio como para el microcontrolador.

Para trabajar en la aplicación, es necesario disponer de Python 3.14 o una versión superior. Asimismo, es necesario instalar las dependencias empleadas durante el desarrollo, las cuales se encuentran especificadas en el proyecto.

Para el desarrollo del firmware del guante se utiliza Arduino IDE junto con el soporte para la placa Arduino Nano ESP32. Además, es necesario instalar las librerías utilizadas para la comunicación con el sensor MPU6050 y, en caso de emplear el modo inalámbrico, las correspondientes a la comunicación WiFi.

El proyecto se encuentra dividido en dos partes independientes: la aplicación de escritorio, desarrollada en Python, y el software del microcontrolador, desarrollado en Arduino. No obstante, ambos componentes se encuentran estrechamente relacionados,

por lo que cualquier modificación realizada en el formato o contenido de los datos enviados por el microcontrolador deberá reflejarse también en la aplicación para garantizar la compatibilidad entre ambos.

Finalmente, para realizar pruebas sobre el sistema completo, será necesario disponer del hardware descrito en el Capítulo 5, así como de una conexión USB o WiFi entre el ordenador y el microcontrolador, según el modo de funcionamiento que se desee utilizar.

A.3. Consideraciones generales sobre el desarrollo

Durante el desarrollo del proyecto se ha mantenido una estructura modular, separando las distintas responsabilidades en archivos independientes. Esta organización facilita el mantenimiento del código y reduce el impacto de futuras modificaciones. Al incorporar nuevas funcionalidades, se recomienda mantener esta separación de responsabilidades, evitando introducir lógica perteneciente a distintos módulos en un mismo archivo. En la Tabla A.1 se resumen los principales módulos de la aplicación y la función que desempeña cada uno de ellos.

Tabla A.1

Principales módulos de la aplicación.

Módulo	Función
app.py	Punto de entrada de la aplicación. Inicializa la interfaz gráfica y coordina el funcionamiento del resto de módulos.
FuncionesLector.py	Procesa las muestras recibidas desde el microcontrolador y extrae las características utilizadas por el sistema.
FuncionesRF.py	Contiene las funciones relacionadas con el entrenamiento del modelo de <i>machine learning</i> , el procesamiento del conjunto de datos y la traducción en tiempo real.
Lector_UI.py	Implementa la funcionalidad del modo de lectura de datos de la aplicación.
FuncionesArduino.py	Gestiona la comunicación con el microcontrolador mediante conexión USB o WiFi.

Por otro lado, cualquier modificación relacionada con los sensores del guante deberá realizarse teniendo en cuenta el proceso de adquisición de datos implementado. Los valores obtenidos por los sensores pueden requerir ajustes o calibraciones para compensar pequeñas diferencias entre dispositivos, por lo que cualquier cambio en el hardware deberá verificarse mediante nuevas capturas de datos. Asimismo, si se modifica el formato o el contenido de los datos enviados por el microcontrolador, será necesario adaptar la aplicación de escritorio para garantizar la compatibilidad entre ambos componentes.

A.4. Instrucciones para construcción

El proyecto dispone de un ejecutable que prepara automáticamente un entorno virtual e instala todas las dependencias del proyecto; sin embargo, si se opta por no utilizar dicho ejecutable, se deben instalar las dependencias que se encuentran en el proyecto y ejecutar el módulo `app.py`.

En el caso del microcontrolador, cualquier modificación debe realizarse mediante Arduino IDE. Tras seleccionar la placa Arduino Nano ESP32 y el puerto de comunicación correspondiente, el código puede verificarse y cargarse directamente en el dispositivo utilizando las opciones proporcionadas por el propio entorno.

Una vez iniciados ambos componentes, el desarrollador podrá comprobar el funcionamiento del sistema y depurar las modificaciones realizadas utilizando las herramientas habituales de Python y Arduino IDE. De este modo, será posible verificar tanto la comunicación entre la aplicación y el microcontrolador como el correcto funcionamiento de las nuevas funcionalidades incorporadas.

B. Manual de usuario

Las instrucciones de uso del software se detallan a continuación. Este manual se dirige al usuario final del software objetivo de este proyecto.

B.1. Introducción

Este manual de usuario tiene como objetivo describir el funcionamiento de la aplicación desarrollada para el guante traductor de lengua de signos. En él se explica cómo utilizar las distintas funcionalidades del sistema, desde la conexión con el guante hasta la adquisición de datos y la traducción de signos.

Asimismo, se incluyen las instrucciones necesarias para la instalación y puesta en marcha de la aplicación, así como una descripción de las principales opciones disponibles para el usuario. El propósito de este capítulo es facilitar el uso del sistema sin necesidad de conocer los detalles técnicos de su implementación.

B.2. Instalación

Para utilizar la aplicación, es necesario disponer de Python 3.14 o una versión superior instalada en el equipo.

El proyecto incluye un archivo ejecutable encargado de preparar automáticamente un entorno virtual e instalar todas las dependencias necesarias para el correcto funcionamiento de la aplicación. Una vez finalizado este proceso, el usuario podrá ejecutar el programa sin necesidad de realizar configuraciones adicionales.

Es recomendable disponer de conexión a Internet durante la primera ejecución, ya que el instalador descargará las bibliotecas para crear el entorno de ejecución.

Por otro lado, el usuario deberá seleccionar el tipo de conexión que utilizará el microcontrolador. Para ello, el proyecto proporciona dos archivos de configuración diferentes, de los cuales deberá cargar el correspondiente en el Arduino mediante el entorno de desarrollo Arduino IDE, en función del modo de comunicación que desee utilizar.

B.3. Uso del sistema

En esta sección se describe el procedimiento para utilizar las distintas funcionalidades de la aplicación. Se explica el proceso de conexión con el microcontrolador, la adquisición de datos para la creación de conjuntos de entrenamiento y el funcionamiento del modo de traducción de signos. Asimismo, se presentan las principales opciones

de la interfaz de usuario y las acciones que debe realizar el usuario para utilizar el sistema de forma correcta.

Configuración de Arduino

Antes de utilizar la aplicación, el usuario deberá cargar en el microcontrolador el archivo de configuración correspondiente al método de conexión que desee utilizar. Para ello, mediante Arduino IDE, deberá seleccionar y cargar el programa destinado a la comunicación por USB, o alternativamente, el destinado a la comunicación WiFi.

Para poder utilizar el programa seleccionado, es necesario instalar las siguientes librerías:

- Adafruit_BusIO
- Adafruit_MPU6050
- Adafruit_Unified_Sensor

Adquisición de datos

Para utilizar esta funcionalidad, el microcontrolador debe estar conectado al equipo mediante el puerto serie. El usuario deberá introducir la etiqueta que identificará el signo que desea registrar y, a continuación, pulsar el botón de inicio para comenzar el proceso de adquisición. Durante este proceso, podrán almacenarse hasta un máximo de 150 muestras para la etiqueta seleccionada.

Las muestras capturadas se almacenan en un archivo independiente asociado a la etiqueta indicada por el usuario. No obstante, aunque la aplicación permite registrar cualquier signo, corresponde al desarrollador revisar posteriormente las muestras obtenidas y decidir cuáles se incorporarán al conjunto de datos empleado para el entrenamiento del modelo de *machine learning*.

Traducción de signos

El modo de traducción permite reconocer en tiempo real los signos realizados con el guante sensorizado. Para ello, el usuario deberá establecer previamente la conexión con el microcontrolador, ya sea mediante USB o WiFi desde la propia aplicación. Dicha elección se corresponde con el programa seleccionado durante la configuración del microcontrolador. Antes de iniciar la traducción, deberá seleccionar el conjunto de datos que se utilizará para entrenar el modelo de *machine learning*.

Una vez seleccionado el conjunto de datos, la aplicación entrena el modelo y queda preparada para comenzar la traducción. A partir de ese momento, los datos enviados por el guante son procesados y clasificados mediante el modelo entrenado, mostrando en la interfaz el signo reconocido en forma de texto junto con varios candidatos y su porcentaje de fiabilidad.

La traducción se realiza de manera continua mientras el modo permanezca activo, permitiendo reconocer sucesivos signos sin necesidad de repetir el proceso de entrenamiento. El usuario puede detener la traducción en cualquier momento mediante la opción correspondiente en la interfaz.

